

P0098R1: Towards Implementation and Use of `memory_order_consume`

Doc. No.: WG21/P0098R1

Date: 2016-01-04

Reply to: Paul E. McKenney, Torvald Riegel, Jeff Preshing,
Hans Boehm, Clark Nelson, Olivier Giroux, and Lawrence Cowl

Email: paulmck@linux.vnet.ibm.com, triegel@redhat.com, jeff@preshing.com
boehm@acm.org, clark.nelson@intel.com, OGiroux@nvidia.com, and Lawrence@Cowl.org

Other contributors: Alec Teal, David Howells, David Lang, George Spelvin,
Jeff Law, Joseph S. Myers, Linus Torvalds, Mark Batty, Michael Matz, Peter Sewell,
Peter Zijlstra, Ramana Radhakrishnan, Richard Biener, Will Deacon, Faisal Vali,
Behan Webster, Tony Tye, JF Bastien, Thomas Koeppel, Jens Maurer, ...

October 16, 2016

This document is a revision of WG21/P0098R0, based on email discussion and on discussions at the 2015 meeting at Kona. This is the final version of this document, which is being released for archival purposes. There will be a follow-on document containing a specific proposal for `memory_order_consume`. WG21/P0098R0 is itself a revision of WG21/N4321, based on email discussions and including two more proposals in Sections 7.9 and 7.11. This proposal has been further refined by discussions on various email reflectors. WG21/N4321 is itself a revision of WG21/N4215, based on feedback at the 2014 UIUC meeting and on the various email reflectors. WG21/N4215 is in turn a revision of WG21/N4036, based on feedback at the 2014 Rapperswil meeting, at the 2014 Redmond SG1 meeting, and on the various email reflectors.

A detailed change log appears starting on page 44.

1 Introduction

The most obscure member of the C11 and C++11 `memory_order` enum seems to be `memory_order_consume` [29]. The purpose of `memory_order_consume` is to allow reading threads to correctly traverse linked data structures without the need for locks, atomic instructions, or (with the exception of DEC Alpha) memory-fence instructions, even though new elements are being inserted into these linked structures before, during, and after the traversal. Without `memory_order_consume`, both the compiler and (again, in the case of DEC Alpha) the CPU would be within their rights to carry out aggressive data-speculation optimizations that would permit readers to see pre-initialization values in the newly added data elements. The purpose of `memory_order_consume` is to prevent these optimizations.

Of course, `memory_order_acquire` may be used as a substitute for `memory_order_consume`, however doing so results in costly explicit memory-fence instructions (or, where available, load-acquire instructions)

on weakly ordered systems such as ARM, Itanium, and PowerPC [3, 9, 12, 13]. These systems enforce dependency ordering in hardware, in other words, if the address used by one memory-reference instruction depends on the value from a preceding load instruction, the hardware forces that earlier load to complete before the later memory-reference instruction commences.¹ Similarly, if the data to be stored by a given store instruction depends on the value from a preceding load instruction, the hardware again forces that earlier load to complete before the later store instruction commences. Recent software tools for ARM and PowerPC can help explicate their memory models [1, 2, 19, 25]. Note that strongly ordered systems like x86, IBM mainframe, and SPARC TSO enforce dependency ordering as a side effect of the fact that they do not reorder loads with subsequent memory references. Therefore, `memory_order_consume` is beneficial on hot code paths, removing the need for hardware ordering instructions for weakly ordered systems and permitting additional compiler optimizations on strongly ordered systems.

When implementing concurrent insertion-only data structures, a few of which are found in the Linux kernel, `memory_order_consume` is all that is required. However, most data structures also require removal of data elements. Such removal requires that the thread removing the data element wait for all readers to release their references to it before reclaiming that element. The traditional way to do this is via garbage collectors (GCs), which have been available for more than half a century [15] and which are now available even for C and C++ [4]. Another way to wait for readers is to use read-copy update (RCU) [21, 24], which explicitly marks read-side regions of code and provides primitives that wait for all pre-existing readers to complete. RCU is gaining significant use both within the Linux kernel [16] and outside of it [5, 6, 8, 14, 30].

Despite the growing number of `memory_order_consume` use cases, there are no known high-performance implementations of `memory_order_consume` loads in any C11 or C++11 environments.

This situation suggests that some change is in order: After all, if implementations do not support the standard's `memory_order_consume` facility, users can be expected to continue to exploit whatever implementation-specific facilities allow them to get their jobs done. This document therefore provides a brief overview of RCU in Section 2 and surveys `memory_order_consume` use cases within the Linux kernel in Section 3. Section 4 looks at how dependency ordering is currently supported in pre-C11 implementations, and then Section 5 looks at possible ways to support those use cases in existing C11 and C++11 implementations, followed by some thoughts on incremental paths towards official support of these use cases in the standards. Section 6 lists some weaknesses in the current C11 and C++11 specification of dependency ordering, and finally Section 7 outlines a few possible alternative dependency-ordering specifications.

Note: SC22/WG14 liaison issue.

2 Introduction to RCU

The RCU synchronization mechanism is often used as a replacement for reader-writer locking because RCU avoids the high-overhead cache thrashing that is characteristic of many common reader-writer-locking implementations. RCU is based on three fundamental concepts:

1. Light-weight in-memory publish-subscribe operation.
2. Operation that waits for pre-existing readers.
3. Maintaining multiple versions of data to avoid disrupting old readers that are still referencing old versions.

These three concepts taken together allow readers and updaters to make forward progress concurrently.

We would like to use C11's and C++11's `memory_order_consume` to implement RCU's lightweight subscribe operation, `rcu_dereference()`. We assume that `rcu_dereference()` is a good example of how developers would exploit the dependency-ordering

¹ But please note that hardware can and does take advantage of the as-if rule, just as compilers do.

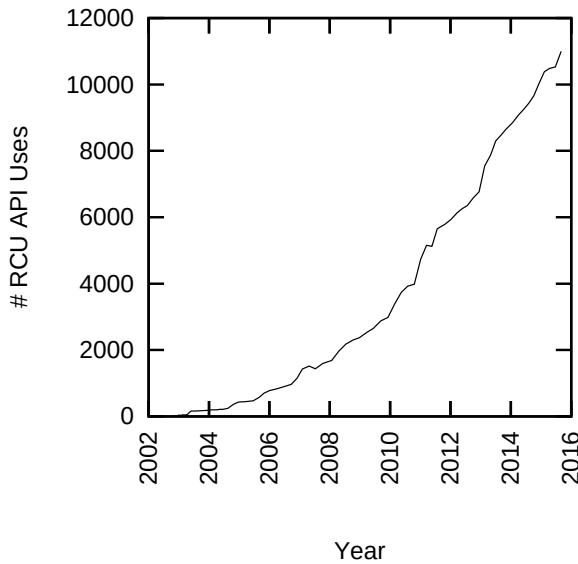


Figure 1: Growth of RCU Usage

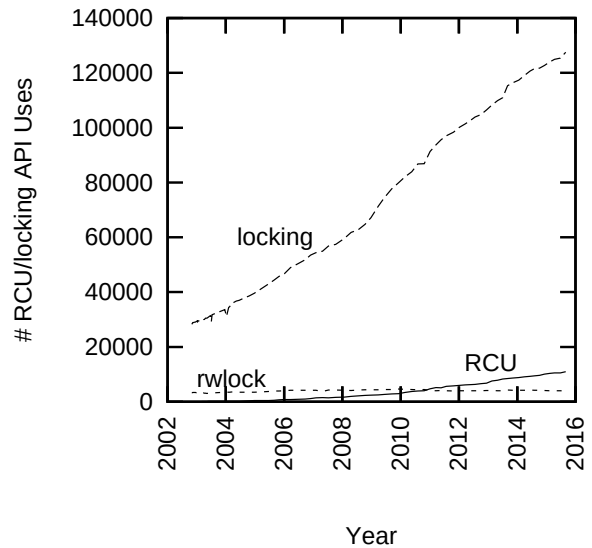


Figure 2: Growth of RCU Usage vs. Locking

feature of weakly ordered systems, so we look to `rcu_dereference()` as an indication of the semantics that `memory_order_consume` should have.

In one typical RCU use case, updaters publish new versions of a data structure while readers concurrently subscribe to whatever version is current at the time a given reader starts. Once all pre-existing readers complete, old versions can be reclaimed. This sort of use case may be a bit unfamiliar to many, but it is extremely effective in many situations, offering excellent performance, scalability, real-time latency, deadlock avoidance, and read-side composability. More details on RCU are readily available [8, 17, 18, 20, 21, 23, 26].

Figure 1 shows the growth of RCU usage over time within the Linux kernel, which is strong evidence of RCU’s effectiveness. However, RCU is a specialized mechanism, so its use is much smaller than general-purpose techniques such as locking, as can be seen in Figure 2. It is unlikely that RCU’s usage will ever approach that of locking because RCU coordinates only between readers and updaters, which means that some other mechanism is required to coordinate among concurrent updates. In the Linux kernel, that

update-side mechanism is normally locking, although pretty much any synchronization mechanism may be used, including transactional memory [10, 11, 28].

However RCU is now being used in many situations where reader-writer locking would be used. Figure 3 shows that the use of reader-writer locking has changed little since RCU was introduced. This data suggests that RCU is at least as important to parallel software as is reader-writer locking.

In more recent years, a user-level library implementation of RCU has been available [7]. This library is now available for many platforms and has been included in a number of Linux distributions. It has been pressed into service for a number of open-source software projects, proprietary products, and research efforts.

Fully and fully performant C11/C++11 support for `memory_order_consume` is therefore quite important. However, good progress can often be made in the short term by focusing on the cases that are commonly used in practice rather than on the general case. The next section therefore takes a rough census of the Linux kernel’s use of the `rcu_dereference()` family of primitives, which `memory_order_consume` is

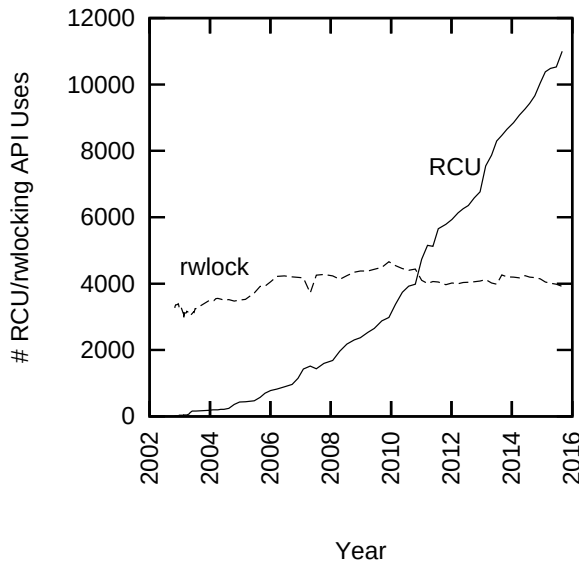


Figure 3: Growth of RCU Usage vs. Reader-Writer Locking

intended to implement.

3 Linux-Kernel Use Cases

Section 3.1 lists types of dependency chains in the Linux kernel, Section 3.2 lists operators used within these dependency chains, Section 3.3 lists operators that are considered to terminate dependency chains, Section 3.4 lists operator that often act as the last link in a dependency chain, and finally Section 3.5 surveys a longer-than-average (but by no means maximal) dependency chain that appears in the Linux kernel.

It is worth reviewing the relationship between `memory_order_acquire` and `memory_order_consume` loads, both of which interact with `memory_order_release` stores.

A `memory_order_release` store is said to *synchronize with* a `memory_order_acquire` load if that load returns the value stored or in some special cases, some later value [29, 1.10p9-1.10p10]. When a `memory_order_release` store synchronizes with a `memory_order_acquire` load, any memory reference preced-

```

1 void new_element(struct foo **pp, int a)
2 {
3     struct foo *p = malloc(sizeof(*p));
4
5     if (!p)
6         abort();
7     p->a = a;
8     atomic_store_explicit(pp, p, memory_order_release);
9 }
10
11 int traverse(struct foo_head *ph)
12 {
13     int a = -1;
14     struct foo *p;
15
16     p = atomic_load_explicit(&ph->h, memory_order_acquire);
17     while (p != NULL) {
18         a = p->a;
19         p = atomic_load_explicit(&p->n, memory_order_acquire);
20     }
21     return a;
22 }
23

```

24

Figure 4: Release/Acquire Linked Structure Traversal

ing the `memory_order_release` store will *happen before* any memory reference following the `memory_order_acquire` load [29, 1.10p13-1.10p14]. This property allows a linked structure to be locklessly traversed by using `memory_order_release` stores when updating pointers to reference new data elements and by using `memory_order_acquire` loads when loading pointers while locklessly traversing the data structure, as shown in Figure 4.

Unfortunately, a `memory_order_acquire` load requires expensive special load instructions or memory-fence instructions on weakly ordered systems such as ARM, Itanium, and PowerPC. Furthermore, in `traverse()`, the address of each `memory_order_acquire` load within the while loop depends on the value of the previous `memory_order_acquire` load.² Therefore, in this case, most weakly ordered systems don't really need the special load instructions or the memory-fence instructions, as these systems can instead rely on the hardware-enforced dependency or-

² The initial load on line 16 might well depend on an earlier load, but for simplicity, this example assumes that the initial `foo_head` structure is statically allocated, and thus not subject to updates.

```

1 void new_element(struct foo **pp, int a)
2 {
3     struct foo *p = malloc(sizeof(*p));
4
5     if (!p)
6         abort();
7     p->a = a;
8     atomic_store_explicit(pp, p, memory_order_release);
9 }
10
11 int traverse(struct foo_head *ph)
12 {
13     int a = -1;
14     struct foo *p;
15
16     p = atomic_load_explicit(&ph->h, memory_order_consume);
17     while (p != NULL) {
18         a = p->a;
19         p = atomic_load_explicit(&p->n, memory_order_consume);
20     }
21     return a;
22 }
23
24

```

Figure 5: Release/Consume Linked Structure Traversal

dering. On strongly ordered systems, `memory_order_acquire` needlessly suppresses memory-movement compiler optimizations.

This is the use case for `memory_order_consume`, which can be substituted for `memory_order_acquire` in cases where hardware dependency ordering applies. One such case is the preceding example, and Figure 5 shows that same example recast in terms of `memory_order_consume`. A `memory_order_release` store is *dependency ordered before* a `memory_order_consume` load when that load returns the value stored, or in some special cases, some later value [29, 1.10p12]. Then, if the load *carries a dependency* to some later memory reference [29, 1.10p11], any memory reference preceding the `memory_order_release` store will happen before that later memory reference [29, 1.10p11-1.10p14]. This means that when there is dependency ordering, `memory_order_consume` gives the same guarantees that `memory_order_acquire` does, but at lower cost.

On the other hand, `memory_order_consume` requires the compiler to track the carries-a-dependency relationships, with the set of such relationships headed by a given `memory_order_consume` load be-

ing called that load’s *dependency chains*. It is quite possible that the complexity of implementing this capability has thus far prevented high-quality `memory_order_consume` implementations from appearing. It is therefore worthwhile to review use of dependency chains in practice in order to determine what types of operations typically appear in dependency chains, which might result in guidance to implementations or perhaps even modifications to the definition of `memory_order_consume`.

3.1 Types of Linux-Kernel Dependency Chains

One goal for `memory_order_consume` is to implement `rcu_dereference()`, which heads a Linux-kernel dependency-ordering tree. There is a number of variants of `rcu_dereference()` in the Linux kernel in order to implement the four flavors of RCU and also to enable RCU usage diagnostics for code that is shared by readers and updaters. These additional variants are `rcu_dereference()`, `rcu_dereference_bh()`, `rcu_dereference_bh_check()`, `rcu_dereference_bh_check()`, `rcu_dereference_check()`, `rcu_dereference_index_check()` (now removed), `rcu_dereference_protected()`, `rcu_dereference_raw()`, `rcu_dereference_sched()`, `rcu_dereference_sched_check()`, `srcu_dereference()`, and `srcu_dereference_check()`. Taken together, there are about 1300 uses of these functions in version 3.13 of the Linux kernel. However, about 250 of those are `rcu_dereference_protected()`, which is used only in update-side code and thus does not head up read-side dependency chains, which leaves about 1000 uses to be inspected for dependency-ordering usage.

3.2 Operators in Linux-Kernel Dependency Chains

A surprisingly small fraction of the possible C operators appear in dependency chains in the Linux kernel, namely `->`, infix `=`, casts, prefix `&`, prefix `*`, `[]`, infix `+`, infix `-`, ternary `?:`, and infix (bitwise) `&`.

By far the two most common operators are the `->` pointer field selector and the `=` assignment oper-

```

1 struct foo {
2     int a;
3 };
4 struct foo *fp;
5 struct foo default_foo;
6
7 int bar()
8 {
9     struct foo *p;
10
11     p = rcu_dereference(fp);
12     return p ? p->a : default_foo.a;
13 }

```

Figure 6: Default Value For RCU-Protected Pointer, Linux Kernel

```

1 struct foo {
2     int a;
3 };
4 std::atomic<foo *> fp;
5 foo default_foo;
6
7 int bar()
8 {
9     foo *p;
10
11     p = fp.load(memory_order_consume);
12     return p ? std::kill_dependency(p->a) : default_foo.a;
13 }

```

Figure 7: Default Value For RCU-Protected Pointer, C++11

ator. Enabling the carries-dependency relationship through only these two operators would likely cover better than 90% of the Linux-kernel use cases.

Casts, the prefix `*` indirection operator, and the prefix `&` address-of operator are used to implement Linux’s list primitives, which translate from list pointers embedded in a structure to the structure itself. These operators are also used to get some of the effects of C++ subtyping in the C language.

The `[]` array-indexing operator, and the infix `+` and `-` arithmetic operators are used to manipulate RCU-protected arrays, as well as to index into arrays contained within RCU-protected structures. RCU-protected arrays are becoming less common because they are being converted into more complex data structures, such as trees. However, RCU-protected structures containing arrays are still fairly common.

The ternary `?:` if-then-else operator is used to handle default values for RCU-protected pointers, for

example, as shown in Figure 6, or in C++11 form in Figure 7. Note that the dependency is carried only through the rightmost two operands of `?:`, never through the leftmost one.

The infix `&` operator is used to mask low-order bits from RCU pointers. These bits are used by some algorithms as markers. Such markers, though not common in the Linux kernel, are well-known in the art, with hazard pointers being but one example [27]. This operator is also sometimes used to locate the beginning of an aligned structure, for example, if `p` references a field within a data structure that is 4096 bytes in size (or smaller), and that is also aligned to a 4096-byte boundary, then `p & ~0xfff` will, with the addition of appropriate casting, produce a pointer to the beginning of the structure. Note that it is expected that both operands of infix `&` are expected to have some non-zero bits, because otherwise a NULL pointer will result, and NULL pointers cannot reasonably be said to carry much of anything, let alone a dependency.

Although I did not find any infix `|` operators in my census of Linux-kernel dependency chains, symmetry considerations argue for also including it, for example, for read-side pointer tagging, or, for another example, locating the beginning of the next in an array of aligned structures. Presumably both of the operands of infix `|` must have at least one zero bit.

To recap, the operators appearing in Linux-kernel dependency chains are: `->`, infix `=`, casts, prefix `&`, prefix `*`, `[]`, infix `+`, infix `-`, ternary `?:`, infix (bitwise) `&`, and probably also `|`.

3.3 Operators Terminating Linux-Kernel Dependency Chains

Although C++11 has the `std::kill_dependency()` function to terminate a dependency chain, no such function exists in the Linux kernel. Instead, Linux-kernel dependency chains are judged to have terminated upon exit from the outermost RCU read-side critical section,³ when existence guarantees are

³ The beginning of a given RCU read-side critical section is marked with `rcu_read_lock()`, `rcu_read_lock_bh()`, `rcu_read_lock_sched()`, or `srcu_read_lock()`, and the end by the corresponding primitive from the list `rcu_read_unlock()`, `rcu_`

handed off from RCU to some other synchronization mechanism (usually locking or reference counting), or when the variable carrying the dependency goes out of scope.

That said, it is possible to analyze Linux-kernel dependency chains to see what part of the chain is actually required by the algorithm in question. We can therefore define the *essential subset* of a dependency chain to be that subset within which ordering is required by the algorithm. In the 3.13 version of the Linux kernel, the following operators always mark the end of the essential subset of a dependency chain: `()`, `!`, `==`, `!=`, `&&`, `||`, infix `*`, `/`, and `%`.

The postfix `()` function-invocation operator is an interesting special case in the Linux kernel. In theory, RCU could be used to protect JITed function bodies, but in current practice RCU is instead used to wait for all pre-existing callers to the function referenced by the previous pointer. The functions are all compiled into the kernel, and the dependency chains are therefore irrelevant to the `()` operator. Hence, in version 3.13 of the Linux kernel, the `()` operator marks the end of the essential subset of any dependency chain that it resides in.

The `!`, `==`, `!=`, `&&`, and `||` operators are used exclusively in “if” statements to make control-flow decisions, and therefore also mark the end of the essential subset of any dependency chains that they reside in. In theory, these relational and boolean operators could be used to form array indexes, but in practice the Linux kernel does not yet do this in RCU dependency chains, and furthermore, as of version 4.2 of the Linux kernel, integers are no longer allowed to carry dependencies except in very restricted situations. The other relational operators (`>`, `<`, `>=`, and `<=`) should probably also be added to this list.

The infix `*`, `/`, and `%` arithmetic operators could potentially be used for construct array addresses, but they are not yet used that way in the Linux kernel. Instead, they are used to do computation on values fetched as the last operation in an essential subset of a dependency chain.

In short, in the current Linux kernel, `()`, `!`, `==`, `!=`, `&&`, `||`, infix `*`, `/`, and `%` all mark the end of the essential subset of a dependency chain. That said, there is potential for them to be used as part of the essential subset of dependency changes in future versions of the Linux kernel. And the same is of course true of the remaining C-language operators, which did not appear within any of the dependency chains in version 3.13 of the Linux kernel.

3.4 Operators Acting as Last Link in Linux-Kernel Dependency Chains

Although the `->` operator is frequently used as part of a Linux-kernel dependency chain, it often is intended to be the last link in that chain. Therefore, the uses cases for the `->` operator deserve special mention.

The first use case involves fetching non-pointer data from an RCU-protected data structure. For example, in the DRDB subsystem in Linux, `->` is used to fetch a timeout value. This code requires that dependency ordering apply to this fetch, but it does not require a dependency chain extending beyond that point. This sort of case would require a `std::kill_dependency()` for implementations based on the C++11 and C11 standards.

The second use case involves linked data structures where an RCU update might be applied on any pointer in the chain, for example, the standard Linux-kernel linked list. The `->` operator provides dependency ordering for the fetch of the `->next` pointer, but that fetch must itself be a `memory_order_consume` load in order to provide the required dependency ordering for the fields in the next structure in the list. Thus, a linked-list traversal consists of a series of back-to-back non-overlapping dependency chains.

These two use cases raise the question of whether a dependency chain can continue beyond a `->` operator. The answer is “yes”, and this occurs when a linked structure is made visible to RCU readers as a unit. For example, consider a linked list where each list element links to a constant binary search tree. If this tree is in place when the element is added to the list, then a `memory_order_consume` load is needed only when fetching the pointer to the element. The

`read_unlock_bh()`, `rcu_read_unlock_sched()`, or `srcu_read_unlock()`. There is currently no C++11 counterpart for an RCU read-side critical section.

```

1 void new_element(struct foo **pp, int a)
2 {
3     struct foo *p = malloc(sizeof(*p));
4
5     if (!p)
6         abort();
7     p->a = a;
8     atomic_store_explicit(pp, p, memory_order_release);
9 }
10
11 int traverse(struct foo_head *ph)
12 {
13     int a = -1;
14     struct foo *p;
15
16     p = atomic_load_explicit(&field_dep(ph, h),
17                             memory_order_consume);
18     while (p != NULL) {
19         a = field_dep(p, a);
20         p = atomic_load_explicit(&field_dep(p, n),
21                                 memory_order_consume);
22     }
23     return a;
24 }

```

Figure 8: Decorated Linked Structure Traversal

dependency chain headed by this fetch suffices to order accesses to the binary search tree.

These cases need to be differentiated. The third use case appears to be the least frequent, which suggests that the `->` operator (or a sequence of `->` operators) always be the last link of a dependency chain. Another alternative is to differentiate on type, so that only pointer-like types (including `intp_t` and `uintp_t`) carry dependencies.

3.5 Linux-Kernel Dependency Chain Length

Many Linux-kernel dependency chains are very short and contained, with a fair number living within the confines of a single C statement. If there were only a few short dependency chains in the Linux kernel, one could imagine decorating all the operators in each dependency chain, for example, replacing the `->` operator with something like the mythical `field_dep()` operator shown on lines 16, 19, and 20 of Figure 8.

However, there are a great many dependency chains that extend across multiple functions. One relatively modest example is in the Linux network stack, in the `arp_process()` function. This depen-

dency chain extends as follows, with deeper nesting indicating deeper function-call levels:

- The `arp_process()` function invokes `__in_dev_get_rcu()`, which returns an RCU-protected pointer. The head of the dependency chain is therefore within the `__in_dev_get_rcu()` function.
- The `arp_process()` function invokes the following macros and functions:
 - `IN_DEV_ROUTE_LOCALNET()`, which expands to the `ipv4_devconf_get()` function.
 - `arp_ignore()`, which in turn calls:
 - * `IN_DEV_ARP_IGNORE()`, which expands to the `ipv4_devconf_get()` function.
 - * `inet_confirm_addr()`, which calls:
 - `dev_net()`, which in turn calls `read_pnet()`.
 - `IN_DEV_ARPFILTER()`, which expands to `ipv4_devconf_get()`.
 - `IN_DEV_CONF_GET()`, which also expands to `ipv4_devconf_get()`.
 - `arp_fwd_proxy()`, which calls:
 - * `IN_DEV_PROXY_ARP()`, which expands to `ipv4_devconf_get()`.
 - * `IN_DEV_MEDIUM_ID()`, which also expands to `ipv4_devconf_get()`.
 - `arp_fwd_pvlan()`, which calls:
 - * `IN_DEV_PROXY_ARP_PVLAN()`, which expands to `ipv4_devconf_get()`.
 - `pneigh_enqueue()`.

Again, although a great many dependency chains in the Linux kernel are quite short, there are quite a few that spread both widely and deeply. We therefore cannot expect Linux kernel hackers to look fondly on any mechanism that requires them to decorate each and every operator in each and every dependency chain as was shown in Figure 8. In fact, even `std::kill_dependency()` will likely be an extremely difficult sell.

4 Dependency Ordering in Pre-C11 Implementations

Pre-C11 implementations of the C language do not have any formal notion of dependency ordering, but these implementations are nevertheless used to build the Linux kernel—and most likely all other software using RCU. This section lays out a few straightforward rules for both implementers (Section 4.2) and users of these pre-C11 C-language implementations (Section 4.1).

4.1 Rules for C-Language RCU Users

The rules for C-language RCU users have evolved over time, so this section will present them in reverse chronological order.

4.1.1 Rules for 2014 GCC Implementations

The primary rule for developers implementing RCU-based algorithms is to avoid letting the compiler determine the value of any variable in any dependency chain. This primary rule implies a number of secondary rules:

1. Use only intrinsic operators on basic types. If you are making use of C++ template metaprogramming or operator overloading, more elaborate rules apply, and those rules are outside the scope of this document.
2. Use a volatile load to head the dependency chain. This is necessary to avoid the compiler repeating the load or making use of (possibly erroneous) prior knowledge of the contents of the memory location, each of which can break dependency chains.
3. Avoid use of single-element RCU-protected arrays. The compiler is within its right to assume that the value of an index into such an array must necessarily evaluate to zero. The compiler could then substitute the constant zero for the computation, breaking the dependency chain and introducing misordering. That said, recent

experience indicates that it is even better to simply avoid carrying dependencies through integer types.

4. Avoid cancellation when using the + and - infix arithmetic operators. For example, for a given variable x , avoid $(x - x)$. The compiler is within its rights to substitute zero for any such cancellation, breaking the dependency chain and again introducing misordering. Similar arithmetic pitfalls must be avoided if the infix *, /, or % operators appear in the essential subset of a dependency chain. Again, recent experience indicates that it is even better to make sure that one of the operands of these arithmetic operators is a pointer-like type.
5. Avoid all-zero operands to the bitwise & operator, and similarly avoid all-ones operands to the bitwise | operator. If the compiler is able to deduce the value of such operands, it is within its rights to substitute the corresponding constant for the bitwise operation. Once again, this breaks the dependency chain, introducing misordering.

Please note that single-bit operands to bitwise & can be dangerous because the compiler requires only a small amount of additional information to deduce the exact value, which could again result in constant substitution. Operands to bitwise | that have only one zero bit are similarly dangerous.

6. If you are using RCU to protect JITed functions, so that the () function-invocation operator is a member of the essential subset of the dependency tree, you may need to interact directly with the hardware to flush instruction caches. This issue arises on some systems when a newly JITed function is using the same memory that was used by an earlier JITed function.
7. Do not use the boolean && and || operators in essential dependency chains. The reason for this prohibition is that they are often compiled using branches. Weak-memory machines such as ARM or PowerPC order stores after such branches, but

can speculate loads, which can break data dependency chains.

8. Do not use relational operators (`==`, `!=`, `>`, `>=`, `<`, or `<=`) in the essential subset of a dependency chain. The reason for this prohibition is that, as for boolean operators, relational operators are often compiled using branches. Weak-memory machines such as ARM or PowerPC order stores after such branches, but can speculate loads, which can break dependency chains.

9. Be very careful about comparing pointers in the essential subset of a dependency chain. As Linus Torvalds explained, if the two pointers are equal, the compiler could substitute the pointer you are comparing against for the pointer in the essential subset of the dependency chain. On ARM and Power hardware, it might be that only the original value carried a hardware dependency, so this substitution would break the chain, in turn permitting misordering. Such comparisons are OK in the following cases:

- (a) The pointer references memory that was initialized at boot time, or otherwise long enough ago that readers cannot still have pre-initialized data cached. Examples include module-init time for module code, before kthread creation for code running in a kthread, while the update-side lock is held, and so on.
- (b) The pointer is never dereferenced after being compared. This exception applies when comparing against the `NULL` pointer or when scanning RCU-protected circular linked lists.
- (c) The pointer being compared against is part of the essential subset of a dependency chain. This can be a different dependency chain, but *only* as long as that chain stems from a pointer that was modified after any initialization of interest. This exception can apply when carrying out RCU-protected traversals from different entry points that converged on the same data structure.

```

1 struct foo {
2     int a;
3 };
4 struct foo *fp;
5 struct foo default_foo;
6
7 int bar()
8 {
9     struct foo *p;
10
11     p = fp;
12     smp_read_barrier_depends();
13     return p ? p->a : default_foo.a;
14 }
```

Figure 9: Default Value For RCU-Protected Pointer, Old Linux Kernel

- (d) The pointer being compared against is fetched using `rcu_access_pointer()` and all subsequent dereferences are stores.
- (e) The pointers compared not-equal *and* the compiler does not have enough information to deduce the value of the pointer. (For example, if the compiler can see that the pointer will only ever take on one of two values, then it will be able to deduce the exact value based on a not-equals comparison.)

The same issue arises when a sequence of inequality comparison operators narrow to a single value.

10. Disable any value-speculation optimizations that your compiler might provide, especially if you are making use of feedback-based optimizations that take data collected from prior runs.

4.1.2 Rules for 2003 GCC Implementations

Prior to the 2.6.9 version of the Linux kernel, there was neither `rcu_dereference()` nor `rcu_assign_pointer()`. Instead, explicit memory barriers were used, `smp_read_barrier_depends()` by readers and `smp_wmb()` by updaters. For example, the code shown for current Linux kernels in Figure 6 would be as shown in Figure 9 for 2.6.8 and earlier versions of the Linux kernel. A similar transformation relates the

older use of `smp_wmb()` and the more recent use of `rcu_assign_pointer()`.

This older API was clearly much more vulnerable to compiler optimizations than is the current API, but the real motivation for this change was readability and maintainability, as can be seen from the commit log for the mid-2004 patch introducing `rcu_dereference()`:

This patch introduced an `rcu_dereference()` macro that replaces most uses of `smp_read_barrier_depends()`. The new macro has the advantage of explicitly documenting which pointers are protected by RCU – in contrast, it is sometimes difficult to figure out which pointer is being protected by a given `smp_read_barrier_depends()` call.

The commit log for the mid-2004 patch introducing `rcu_assign_pointer()` justifies the change in terms of eliminating hard-to-use explicit memory barriers:

Attached is a patch that adds an `rcu_assign_pointer()` that allows a number of explicit `smp_wmb()` memory barriers to be dispensed with, improving readability.

The importance of suppressing compiler optimizations did not become apparent until much later. In fact, a volatile cast was not added to the implementation of `rcu_dereference()` until 2.6.24 in early 2008.

4.1.3 Rules for 1990s Sequent C Implementations

1990s systems featured far slower CPUs and much less memory that is commonly provisioned today, and the compilers were correspondingly less sophisticated. Therefore, at that time, a simple C-language field selector was used instead of any sort of `rcu_dereference()` or `memory_order_consume` operation. Not only was there no volatile cast, there also was nothing resembling `smp_read_barrier_depends()`. The lack of `smp_read_barrier_depends()` is not too surprising, given that DYNIX/ptx did not run on DEC Alpha.

```

1 void new_element(struct foo **pp, int a)
2 {
3     struct foo *p = malloc(sizeof(*p));
4
5     if (!p)
6         abort();
7     p->a = a;
8     rcu_assign_pointer(pp, p);
9 }
10
11 int traverse(struct foo_head *ph)
12 {
13     int a = -1;
14     struct foo *p;
15
16     p = rcu_dereference(&ph->h);
17     while (p != NULL) {
18         if (p == (struct foo *)0xbadfabe)
19             a = ((struct foo *)0xbadfabe)->a;
20         else
21             a = p->a;
22         p = rcu_dereference(&p->n);
23     }
24     return a;
25 }

```

Figure 10: Dangerous Optimizations: Hardware Branch Predictions

This approach was nevertheless quite reliable because the use cases within the DYNIX/ptx kernel were both few and straightforward, and provided little or no opportunity for optimizations that might break dependency chains.

4.2 Rules for C-Language Implementers

The main rule for C-language implementers is to avoid any sort of value speculation, or, at the very least, provide means for the user to disable such speculation. An example of a value-speculation optimization that can be carried out with the help of hardware branch prediction is shown in Figure 10, which is an optimized version of the code in Figure 5. This sort of transformation might result from feedback-directed optimization, where profiling runs determined that the value loaded from `ph` was almost always `0xbadfabe`. Although this transformation is correct in a single-threaded environment, in a concurrent environment, nothing stops the compiler or the CPU from speculating the load on line 19 before it executes the `rcu_dereference()` on line 16, which

could result in line 19 executing before the corresponding store on line 7, resulting in a garbage value in variable `a`.⁴

There *are* some situations where this sort of optimization would be safe, including:

1. The value speculated is a numeric value rather than a pointer, so that if the guess proves correct after the fact, the computation will be appropriate after the fact.
2. The value speculated is a pointer to invariant data, then reasonable values will be produced by dereferencing, even if the guess proves to have been correct only after the fact.
3. As above, but where any updates result in data that produces appropriate computations at any and all phases of the update.

However, this list does not contain the general case of `memory_order_consume` loads.

Pure hardware implementations of value speculation can avoid this problem because they monitor cache-coherence protocol events that would result from some other CPU invalidating the guess.

In short, compiler writers must provide means to disable all forms of value speculation, unless the speculation is accompanied by some means of detecting the race condition that Figure 10 is subject to.

Are there other dependency-breaking optimizations that should be called out separately?

5 Dependency Ordering in C11 and C++11 Implementations

The simplest way to avoid dependency-ordering issues is to strengthen all `memory_order_consume` operations to `memory_order_acquire`. This functions correctly, but may result in unacceptable performance due to memory-barrier instructions on weakly ordered systems such as ARM and PowerPC,⁵ and may

⁴ Kudos to Olivier Giroux for pointing out use of branch prediction to enable value speculation.

⁵ From a Linux-kernel community viewpoint, that should read “*will* result in unacceptable performance”.

```

1 int a(struct foo *p [[carries_dependency]])
2 {
3     return std::kill_dependency(p->a != 0);
4 }
5
6 int b(int x)
7 {
8     return x;
9 }
10
11 foo *c()
12 {
13     return fp.load(memory_order_consume);
14     /* return rcu_dereference(fp) in Linux kernel. */
15 }
16
17 int d()
18 {
19     int a;
20     foo *p;
21
22     rcu_read_lock();
23     p = c();
24     a = p->a;
25     rcu_read_unlock();
26     return a;
27 }

```

Figure 11: Example Functions for Dependency Ordering, Part 1

further unnecessarily suppress code-motion optimizations.

Another straightforward approach is to avoid value speculation and other dependency-breaking optimizations. This might result in missed opportunities for optimization, but avoids any need for dependency-chain annotations and also all issues that might otherwise arise from use of dependency-breaking optimizations. This approach is fully compatible with the Linux kernel community’s current approach to dependency chains. Unfortunately, there are any number of valuable optimizations that break dependency chains, so this approach seems impractical.

A third approach is to avoid value speculation and other dependency-breaking optimizations in any function containing either a `memory_order_consume` load or a `[[carries_dependency]]` attribute. For example, the hardware-branch-prediction optimization shown in Figure 10 would be prohibited in such functions, as would cancellation optimizations such as optimizing `a = b + c - c` into `a = b`. This too

```

1 [[carries_dependency]] struct foo *e()
2 {
3     return fp.load(memory_order_consume);
4     /* return rcu_dereference(fp) in Linux kernel. */
5 }
6
7 int f()
8 {
9     int a;
10    foo *p;
11
12    rcu_read_lock();
13    p = e();
14    a = p->a;
15    rcu_read_unlock();
16    return std::kill_dependency(a);
17 }
18
19 int g()
20 {
21     int a;
22     foo *p;
23
24     rcu_read_lock();
25     p = e();
26     a = p->a;
27     rcu_read_unlock();
28     return b(a);
29 }

```

Figure 12: Example Functions for Dependency Ordering, Part 2

can result in missed opportunities for optimization, though very probably many fewer than the previous approach. This approach can also result in issues due to dependency-breaking optimizations in functions lacking `[[carries_dependency]]` attributes, for example, function `d()` in Figure 11. It can also result in spurious memory-barrier instructions when a dependency chain goes out of scope, for example, with the `return` statement of function `g()` in Figure 12.

A fourth approach is to add a compile-time operation corresponding to the beginning and end of RCU read-side critical section. These would need to be evaluated at compile time, taking into account the fact that these critical sections can nest and can be conditionally entered and exited. Note that the exit from an outermost RCU read-side critical section should imply a `std::kill_dependency()` operation on each variable that is live at that point in the code.⁶

⁶ What if a given `rcu_read_unlock()` sometimes marked the end of an outermost RCU read-side critical section, but other times was nested in some other RCU read-side critical section?

Although it is probably impossible to precisely determine the bounds of a given RCU read-side critical section in the general case, conservative approaches that might overestimate the extent of a given section should be acceptable in almost all cases. This approach would make functions `c()` and `d()` in Figure 11 handle dependency chains in a natural manner, but avoiding whole-program analysis would require something similar to the `[[carries_dependency]]` annotations called out in the C11 and C++11 standards.

A fifth approach would be to require that all operations on the essential subset of any dependency chain be annotated. This would greatly ease implementation, but would not be likely to be accepted by the Linux kernel community.

A sixth approach is to track dependencies as called out in the C11 and C++11 standards. However, instead of emitting a memory-barrier instruction when a dependency chain flows into or out of a function without the benefit of `[[carries_dependency]]`, insert an implicit `std::kill_dependency()` invocation. Implementation should also optionally issue a diagnostic in this case. The motivation for this approach is that it is expected that many more `kill_dependencies()` than `[[carries_dependency]]` would be required to convert the Linux kernel's RCU code to C11. In the example in Figure 12, this approach would allow function `g()` to avoid emitting an unnecessary memory-barrier instruction, but without function `f()`'s explicit `std::kill_dependency()`. Both functions are in Figure 12.

A seventh and final approach is to track dependencies as called out in the C11 and C++11 standards. With this approach, functions `e()` and `f()` properly preserve the required amount of dependency ordering.

In that case, there should be no `std::kill_dependency()`.

```

1 p = atomic_load_explicit(gp, memory_order_consume);
2 if (p == ptr_a)
3   a = p->special_a;
4 else
5   a = p->normal_a;

```

Figure 13: Dependency-Ordering Value-Narrowing Hazard

6 Weaknesses in C11 and C++11 Dependency Ordering

Experience has shown several weaknesses in the dependency ordering specified in the C11 and C++11 standards:

1. The C11 standard does not provide attributes, and in particular, does not provide the `[[carries_dependency]]` attribute. This prevents the developer from specifying that a given dependency chain passes into or out of a given function. The usual C-language response to this situation is to introduce a new keyword.
2. The implementation complexity of the dependency-chain tracking required by both standard can be quite onerous on the one hand, and the overhead of unconditionally promoting `memory_order_consume` loads to `memory_order_acquire` can be excessive on weakly ordered implementations on the other. There is therefore no easy way out for a `memory_order_consume` implementation on a weakly ordered system.
3. The function-level granularity of `[[carries_dependency]]` seems too coarse. One problem is that points-to analysis is non-trivial, so that compilers are likely to have difficulty determining whether or not a given pointer carries a dependency. For example, the current wording of the standard (intentionally!) does not disallow dependency chaining through stores and loads. Therefore, if a dependency-carrying value might ever be written to a given variable, an implementation might reasonably assume that *any* load

from that variable must be assumed to carry a dependency.

4. The rules set out in the standard [29, 1.10p11] do not align well with the rules that developers must currently adhere to in order to maintain dependency chains when using pre-C11 and pre-C++11 compilers (see Section 4.1). For example, the standard requires `(x-x)` to carry a dependency, and providing this guarantee would at the very least require the compiler to also turn off optimizations that remove `(x-x)` (and similar patterns) if `x` might possibly be carrying a dependency. For another example, consider the value-speculation-like code shown in Figure 13 that is sometimes written by developers, and that was described in bullet 9 of Section 4.1. In this example, the standard requires dependency ordering between the `memory_order_consume` load on line 1 and the subsequent dereference on line 3, but a typical compiler would not be expected to differentiate between these two apparently identical values. These two examples show that a compiler would need to detect and carefully handle these cases either by artificially inserting dependencies, omitting optimizations, differentiating between apparently identical values, or even by emitting `memory_order_acquire` fences.
5. The whole point of `memory_order_consume` and the resulting dependency chains is to allow developers to optimize their code. Such optimization attempts can be completely defeated by the `memory_order_acquire` fences that the standard currently requires when a dependency chain goes out of scope without the benefit of a `[[carries_dependency]]` attribute. Preventing the compiler from emitting these fences requires liberal use of `std::kill_dependency()`, which clutters code, requires large developer effort, and further requires that the developer know quite a bit about which code patterns a given version of a given compiler can optimize (thus avoiding needless fences) and which it cannot (thus requiring manual insertion of `std::kill_dependency()`).

As of this writing, no known implementations fully support C11 or C++11 dependency ordering.

It is worth asking why Paul didn't anticipate these weaknesses. There are several reasons for this:

1. Compiler optimizations have become more aggressive over the eight years since Paul started working on standardization.
2. New dependency-ordering use cases have arisen during that same time, in particular, there are longer dependency chains and more of them, including dependency chains spanning multiple compilation units.
3. The number of dependency chains has increased by more than an order of magnitude during that time, so that changes in code style can be expected to face a commensurate increase in resistance from the Linux kernel community – unless those changes bring some tangible benefit.

With that, let's look at some potential alternatives to dependency ordering as defined in the C11 and C++11 standards.

7 Potential Alternatives to C11 and C++11 Dependency Ordering

Given the weaknesses in the current standard's specification of dependency ordering, it is quite reasonable to consider alternatives. To this end, Section 7.1 discusses ease-of-use issues involved with revisions to the C11 and C++11 definitions of dependency ordering, Section 7.2 enlists help from the type system, but also imposes value restrictions (thus revising the C11 and C++11 semantics for dependencies), Section 7.3 enlists help from the type system without the value restrictions, and Section 7.4 describes a whole-program approach to dependency chains (also revising the C11 and C++11 semantics for dependencies). Section 7.5 describes a post-Rapperswil proposal that dependency chains be restricted to function-scope local variables and temporaries, and

Section 7.6 describes a second post-Rapperswil proposal that the `[[carries_dependency]]` attribute be used to label local-scope variables that carry dependencies. Section 7.7 describes a proposal discussed verbally at Rapperswil that explicitly marks the tails of dependency chains. Section 7.8 describes the inverse, namely marking the heads of dependency chains. Section 7.9 describes an approach that avoids marking by sharply restricting the number and type of operations permitted in dependency chains. Each approach appears to have advantages and disadvantages, so it is hoped that further discussion will either help settle on one of these alternatives or generate something better. To help initiate this discussion, Section 7.11 uses a new `_Carries_dependency` storage class to mark objects that carry dependencies. Section 7.12 provides an initial comparative evaluation.

7.1 Revising C11 and C++11 Dependency-Ordering Definition

The following sections each describe a proposed revision of the dependency-ordering definition from that in the current C11 and C++11 standards. In many of these proposals, developers are required to follow an additional rule in order to be able to rely on dependency ordering: Subsequent execution must not lead to a situation where there is only one possible value for the variable that is intended to carry the dependency.⁷ This is shown in Figure 17, where the compiler is permitted to break dependency ordering on line 6 because it knows that the value of `p` is equal to that of `q`, which means that it could substitute the latter value from the former, which would break dependency ordering. In short, a dependency chain breaks if it comes to a point where only a single value is possible, regardless of the value of the `memory_order_consume` load heading up the chain. At first glance, this additional rule could be quite difficult to live with, as dependency ordering could come and go

⁷ This restricted notion of dependence is sometimes called *semantic dependence*, and the value at the end of a dependence chain that does not represent a semantic dependence is sometimes said to be *independent* of the value at the head of the dependency chain.

```

1 int my_array[MY_ARRAY_SIZE];
2
3 i = atomic_load_explicit(gi, memory_order_consume);
4 r1 = my_array[i];

```

Figure 14: Single-Element Arrays and Dependency Ordering

depending on small details of code far away from that point in the dependency chain.

However, a review of the Linux-kernel operators in Section 3.2 shows that the most commonly used operators act identically under both definitions. The problem-free operators include $\rightarrow$, infix $=$, casts, prefix $\&$, prefix $*$, and ternary $?:$.

One example of a potentially troublesome operator, namely $==$, is shown in Figure 17, where line 6 breaks dependency ordering because the value of p is known to be equal to that of q , which is not part of a dependency chain. This example could be addressed through careful diagnostic design coupled with appropriate coding standards. For example, the compiler could emit a warning on line 6, but remain silent for the equivalent line substituting q for p , namely, `do_something_with(q->a)`.

Another example is the use of postfix $[]$ that is shown in Figure 14. If this code fragment was compiled with `MY_ARRAY_SIZE` equal to one, there is no dependency ordering between lines 3 and 4, but that same code fragment compiled with `MY_ARRAY_SIZE` equal to two or greater *would* be dependency-ordered. Here a diagnostic for single-element arrays might prove useful, and such a diagnostic can easily be supplied in this case using `#if` and `#error`. Or, better yet, don't carry dependencies through through integers for use as array indexes.

In the Linux kernel, infix $+$ and $-$ are used for pointer and array computations. These are all safe in that they operate on an integer and pointer, so that any cancellation will not normally be detectable at compile time. However, one big purpose of diagnostics is to detect abnormal conditions indicating probable bugs. Therefore, in cases where the compiler can determine that two values from dependency chains are annihilating each other via infix $+$ and $-$, a diagnostic would be appropriate.

```

1 struct liststackhead {
2     struct liststack __rcu *first;
3 };
4
5 struct liststack {
6     struct liststack __rcu *next;
7     void *t;
8     struct rcu_head rh;
9 };
10
11 _Carries_dependency
12 void *ls_front(struct liststackhead *head)
13 {
14     _Carries_dependency void *data;
15     struct liststack *lsp;
16
17     rcu_read_lock();
18     lsp = rcu_dereference(head->first);
19     if (lsp == NULL)
20         data = NULL;
21     else
22         data = rcu_dereference(lsp->t);
23     rcu_read_unlock();
24     return data;
25 }

```

Figure 15: List-Based-Stack Example Code, 1 of 2

Similarly, the Linux kernel uses infix (bitwise) $\&$ to manipulate bits at the bottom of a pointer, where again cancellation will not normally be detectable at compile time—except in the case of operations on a NULL pointer, for which dependency ordering is not meaningful in any case. However, as with infix $+$ and $-$, if the compiler detects value annihilation, a diagnostic would be appropriate.

Although issues with false positives and negatives needs further investigation, there is reason to hope that this revision of the definition of dependency ordering might avoid significant impacts on ease of use. With this hope, we proceed to the specific proposals, using the code in Figures 15 and 16 to show some sample code using Linux-kernel nomenclature with the addition of a mythical C keyword `_Carries_dependency` to annotate parameters, variables, and return values that carry dependencies. Please note that this code example in no way endorses the dubious practice of creating a parallel program with the sort of choke point exemplified by the head of this list. Note also that `cmpxchg()` heads a dependency chain, which is completely reasonable within the context of the Linux kernel due to its acquire semantics, which of course might be argued to indicate that the


```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3     struct liststack *lsp;
4     struct liststack *lsnp1;
5     struct liststack *lsnp2;
6     size_t sz;
7
8     sz = sizeof(*lsp);
9     sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10    sz *= CACHE_LINE_SIZE;
11    lsp = malloc(sz);
12    if (!lsp)
13        return -ENOMEM;
14    if (!t)
15        abort();
16    lsp->t = t;
17    rcu_read_lock();
18    lsnp2 = ACCESS_ONCE(head->first);
19    do {
20        lsnp1 = lsnp2;
21        lsp->next = lsnp1;
22        lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23    } while (lsnp1 != lsnp2);
24    rcu_read_unlock();
25    return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30     struct liststack *lsp;
31
32     lsp = container_of(rhp, struct liststack, rh);
33     free(lsp);
34 }
35
36 _Carries_dependency
37 void *ls_pop(struct liststackhead *head)
38 {
39     _Carries_dependency struct liststack *lsp;
40     struct liststack *lsnp1;
41     _Carries_dependency struct liststack *lsnp2;
42     _Carries_dependency void *data;
43
44     rcu_read_lock();
45     lsnp2 = rcu_dereference(head->first);
46     do {
47         lsnp1 = lsnp2;
48         if (lsnp1 == NULL) {
49             rcu_read_unlock();
50             return NULL;
51         }
52         lsp = rcu_dereference(lsnp1->next);
53         lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54     } while (lsnp1 != lsnp2);
55     data = rcu_dereference(lsnp2->t);
56     rcu_read_unlock();
57     call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58     return data;
59 }

```

Figure 16: List-Based-Stack Example Code, 2 of 2

```

1 value_dep_preserving struct foo *p;
2
3 p = atomic_load_explicit(gp, memory_order_consume);
4 q = some_other_pointer;
5 if (p == q)
6     do_something_with(p->a);
7 else
8     do_something_else_with(p->b);

```

Figure 17: Single-Value Variables and Dependency Ordering

annotations in `ls_pop()` are unnecessary.

7.2 Type-Based Designation of Dependency Chains With Restrictions

This approach was formulated by Torvald Riegel in response to Linus Torvalds’s spirited criticisms of the current C11 and C++11 wording.

This approach introduces a new `value_dep_preserving` type qualifier. Dependency ordering is preserved only via variables having this type qualifier. This is meant to model the real scope of dependencies, which is data flow, not execution at function-level granularity. This approach should therefore give developers much finer control of which dependencies are tracked.

Assigning from a `value_dep_preserving` value to a non-`value_dep_preserving` variable terminates the tracking of dependencies in much the same way that an explicit `std::kill_dependency()` would. However, unlike an explicit `std::kill_dependency()`, compilers should be able to emit a suppressable warning on implicit conversions, so as to alert the developer about otherwise silent dropping of dependency tracking.⁸

Next, we specify that `memory_order_consume` loads return a `value_dep_preserving` type by default; the compiler must assume such a load to be capable of producing any value of the underlying type. In other words, the implementation is not permitted to apply any value-restriction knowledge it might gain from

⁸ Other choices are possible in this case, including emitting a `memory_order_acquire` fence in order to conservatively preserve a potentially intended ordering.

whole-program analysis. We call this a *local semantic dependency* to distinguish not only from a pure (syntactic) dependency, but also from a *global semantic dependency*, where global information may be applied. Note that any global semantic dependency is also a local semantic dependency, but that any local semantic dependency which is headed by a variable that can be proven to take on only a single value is *not* a global semantic dependency. The term “semantic dependency” should be interpreted to mean a global semantic dependency unless otherwise stated.

This allows developers to start with a clean slate for the additional rule that they must follow to be able to rely on dependency ordering: Subsequent execution must not lead to a situation there is only one possible value for the `value_dep_preserving` expression, because otherwise the implementation is permitted to break the dependency chain. As noted earlier, this is shown in Figure 17, where the compiler is permitted to break dependency ordering on line 6 because it knows that the value of `p` is equal to that of `q`, which means that it could substitute the latter value from the former, which would break dependency ordering.

This approach has several advantages:

1. The implementation is simpler because no dependency chains need to be traced. The implementation can instead drive optimization decisions strictly from type information.
2. Use of the `value_dep_preserving` type modifier allows the developer to limit the extent of the dependency chains.
3. This type modifier can be used to mark a dependency chain’s entry to and exit from a function in a straightforward way, without the need for attributes.
4. The `value_dep_preserving` type modifiers serve as valuable documentation of the developer’s intent.
5. This approach permits many additional optimizations compared to those permitted by the current standard on code that carries a dependency. Expressions such as `(x-x)` no longer

require establishment of artificial dependencies and the compiler is no longer required to detect value-narrowing hazards like that shown in Figure 13. However, the compiler is still prohibited from adding its own value-speculation optimizations.

6. Linus Torvalds seems to be OK with it, which indicates that this set of rules might be practical from the perspective of developers who currently exploit dependency chains.

According to Peter Sewell, one disadvantage is that this approach will be quite difficult to model, which in turn will pose obstacles for the analysis tooling that will be increasingly necessary for large-scale concurrent programming efforts. In particular, the concern is that forcing the compiler to assume that a `memory_order_consume` load could possibly return any value permitted by its type might require program-analysis tools to consider counterfactual hypothetical executions, which might complicate specification of semantics and verification.

Figures 18 and 19 show how this approach plays out with the list-based stack.

7.3 Type-Based Designation of Dependency Chains

Jeff Preshing made an off-list suggestion of using a `value_dep_preserving` type modifier as suggested by Torvald Riegel, but using this type modifier to strictly enforce dependency ordering. For example, consider the code fragment shown in Figure 17. The scheme described in Section 7.2 would *not* necessarily enforce dependency ordering between the load on line 3 and the access one line 6, while the approach described in this section would enforce dependency ordering in this case.

Furthermore, cancelling or value-destruction operations on `value_dep_preserving` values would *not* disrupt dependency ordering. As with the current C11 and C++11 standards, the implementation would be required to emit a memory-barrier instruction or compute an artificial dependency for such operations. (Note however that use of cancelling or

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack value_dep_preserving *first;
6 };
7
8 struct liststack {
9   struct liststack value_dep_preserving *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 value_dep_preserving
15 void *ls_front(struct liststackhead *head)
16 {
17   value_dep_preserving void *data;
18   value_dep_preserving struct liststack *lsp;
19
20   rcu_read_lock();
21   lsp = rcu_dereference(head->first);
22   if (lsp == NULL)
23     data = NULL;
24   else
25     data = rcu_dereference(lsp->t);
26   rcu_read_unlock();
27   return data;
28 }

```

Figure 18: List-Based-Stack Restricted Type-Based Designation, 1 of 2

value-destruction operations on dependency chains has proven quite rare in practice.)

This approach shares many of the advantages of Torvald Riegel’s approach:

1. The implementation is simpler because no dependency chains need be traced. The implementation can instead drive optimization decisions strictly from type information.
2. Use of the `value_dep_preserving` type modifier allows the developer to limit the extent of the dependency chains.
3. This type modifier can be used to mark a dependency chain’s entry to and exit from a function in a straightforward way, without the need for attributes.
4. The `value_dep_preserving` type modifiers serve as valuable documentation of the developer’s intent.

```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3   struct liststack *lsp;
4   struct liststack *lsnp1;
5   struct liststack *lsnp2;
6   size_t sz;
7
8   sz = sizeof(*lsp);
9   sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10  sz *= CACHE_LINE_SIZE;
11  lsp = malloc(sz);
12  if (!lsp)
13    return -ENOMEM;
14  if (!t)
15    abort();
16  lsp->t = t;
17  rcu_read_lock();
18  lsnp2 = ACCESS_ONCE(head->first);
19  do {
20    lsnp1 = lsnp2;
21    lsp->next = lsnp1;
22    lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23  } while (lsnp1 != lsnp2);
24  rcu_read_unlock();
25  return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30   struct liststack *lsp;
31
32   lsp = container_of(rhp, struct liststack, rh);
33   free(lsp);
34 }
35
36 value_dep_preserving
37 void *ls_pop(struct liststackhead *head)
38 {
39   value_dep_preserving struct liststack *lsp;
40   struct liststack *lsnp1;
41   value_dep_preserving struct liststack *lsnp2;
42   value_dep_preserving void *data;
43
44   rcu_read_lock();
45   lsnp2 = rcu_dereference(head->first);
46   do {
47     lsnp1 = lsnp2;
48     if (lsnp1 == NULL) {
49       rcu_read_unlock();
50       return NULL;
51     }
52     lsp = rcu_dereference(lsnp1->next);
53     lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54   } while (lsnp1 != lsnp2);
55   data = rcu_dereference(lsnp2->t);
56   rcu_read_unlock();
57   call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58   return data;
59 }

```

Figure 19: List-Based-Stack Restricted Type-Based Designation, 2 of 2

5. Although optimizations on a dependency chain are restricted just as in the current standard, the use of `value_dep_preserving` restricts the dependency chains to those intended by the developer.
6. Restricting dependency-breaking optimizations on all dependency chains marked `value_dep_preserving`, without exceptions for cases in which the compiler knows too much, might make this approach easier to learn and to use.

It is expected that modeling this approach should be straightforward because the modeling tools would be able to make use of the type information. This approach results in the same code as shown in Figures 18 and 19 of the previous section.

7.4 Whole-Program Option

This approach, also suggested off-list by Jeff Preshing, has the goal of reusing existing non-dependency-ordered source code unchanged (albeit requiring recompilation in most cases).⁹ For example, this approach permits an instance of `std::map` to be referenced by a pointer loaded via `memory_order_consume` and to provide that `std::map` instance with the benefits of dependency ordering without any code changes whatsoever to `std::map`. It is important to note that this protection will be provided only to a read-only `std::map` that is referenced by a changing pointer loaded via `memory_order_consume`, in particular, *not* to a concurrently updated `std::map` referenced by a pointer (read-only or otherwise) loaded via `memory_order_consume`. This latter case *would* require changes to the underlying `std::map` implementation, at a minimum, changing some of the loads to be `memory_order_consume` loads. Nevertheless, the ability to provide dependency-ordering protection to pre-existing linked data structures is valuable, even with this read-only restriction.

This approach, which again does require full recompilation, can be implemented using two approaches:

1. Promote all `memory_order_consume` loads to `memory_order_acquire`, as may be done with the current standard.
2. On architectures that respect memory ordering, prohibit all dependency-breaking optimizations throughout the entire program, but only in cases where a change in the value returned by a `memory_order_consume` load could cause a change in the value computed later in that same dependency chain, in other words, where there is a global semantic dependency. Note again that the possibility of storing a value obtained from a `memory_order_consume` load, then loading it later, means that normal loads as well as `memory_order_relaxed` loads often must be considered to head their own dependency chains, but only when loaded by the same thread that did the store.

Some implementations might allow the developer to choose between these two approaches, for example, by using a compiler switch provided for that purpose.

This approach also has the effect of permitting a trivial implementation of a `memory_order_consume atomic_thread_fence()`. When using the first implementation approach, the `atomic_thread_fence()` is simply promoted to `memory_order_acquire`. Interestingly enough, when using the second approach, the `memory_order_consume atomic_thread_fence()` may simply be ignored. The reason for this is that this approach has the effect of promoting `memory_order_relaxed` loads to `memory_order_consume`, which already globally enforces all the ordering that the `memory_order_consume atomic_thread_fence()` is required to provide locally.¹⁰

This approach has its own set of advantages and disadvantages:

1. This approach dispenses with the `[[carries_dependency]]` attribute and the `std::kill_dependency()` primitive.

¹⁰ Of course, this presumed promotion from `memory_order_relaxed` to `memory_order_consume` means that architectures such as DEC Alpha that do not respect dependency ordering must continue to use the first option of emitting memory-ordering instructions for `memory_order_consume` loads.

⁹ A module or library that is known to never carry a dependency need not be recompiled.

2. This approach better promotes reuse of existing source code. In particular, it should require no changes to the current Linux-kernel source base, aside from changes to the `rcu_dereference()` family of primitives.
3. This approach allows implementations to carry out dependency-breaking optimizations on dependency chains as long as a change in the value from the `memory_order_consume` load does not change values further down the dependency chain, both with and without the optimization. Jeff conjectures that the set of dependency-breaking optimizations used in practice apply only outside of dependency chains, by the revised definition in which single-value restrictions break dependency chains.¹¹ If this conjecture holds, it also applies to Torvald's approach described in Section 7.2.
4. Code that follows the rules presented in Section 4.1 (substituting `memory_order_consume` loads for `volatile` loads) would have its dependency ordering properly preserved.

It is unlikely that this approach could be modeled reasonably given the current state of the art. The requirement that any given `memory_order_consume` load be able to generate at least two different values at the tail of the dependency chain is believed to be a show-stopper, especially when coupled with whole-program analysis, which might find that there is only one value entering at the head of the dependency chain.

This approach allows annotations to be discarded, as shown in Figures 20 and 21. However, the `memory_order_consume` loads are still required in order to enable the promote-to-acquire implementation style.

7.5 Local-Variable Restriction

This approach, suggested off-list by Hans Boehm, limits the extent of dependency trees to a local, which includes local variables, temporaries, function arguments, and return variables. Assigning a value from a

¹¹ This is certainly the case for the usual optimizations exemplified by replacing `(x-x)` with zero.

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack *first;
6 };
7
8 struct liststack {
9   struct liststack *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 void *ls_front(struct liststackhead *head)
15 {
16   void *data;
17   struct liststack *lsp;
18
19   rcu_read_lock();
20   lsp = rcu_dereference(head->first);
21   if (lsp == NULL)
22     data = NULL;
23   else
24     data = rcu_dereference(lsp->t);
25   rcu_read_unlock();
26   return data;
27 }

```

Figure 20: List-Based-Stack Whole-Program Approach, 1 of 2

`memory_order_consume` load to such an object begins a dependency chain. Assigning a value loaded from such a local to a global variable (including function-local variables marked `static`) or to the heap implies a `std::kill_dependency()`, so that dependency chains are confined to locals. However, if the compiler is unable to see the full dependency chain, for example, because it passes into a function in another translation unit that is not marked `[[carries_dependency]]`, the compiler should promote `memory_order_consume` to `memory_order_acquire`.¹²

Section 3.2 indicates that the following operators should transmit dependency status from one local variable or temporary to another: `->`, infix `=`, casts, prefix `&`, prefix `*`, `[]`, infix `+`, infix `-`, ternary `?:`, infix (bitwise) `&`, and probably also `|`. Similarly, Section 3.3 indicates that the following operators should imply a `std::kill_dependency()`: `()`, `!`, `==`, `!=`, `&&`, `||`, infix `*`, `/`, and `%`.

¹² Some implementations might provide means to allow the user to specify that a diagnostic be generated if such promotion is necessary.

```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3     struct liststack *lsp;
4     struct liststack *lsnp1;
5     struct liststack *lsnp2;
6     size_t sz;
7
8     sz = sizeof(*lsp);
9     sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10    sz *= CACHE_LINE_SIZE;
11    lsp = malloc(sz);
12    if (!lsp)
13        return -ENOMEM;
14    if (!t)
15        abort();
16    lsp->t = t;
17    rcu_read_lock();
18    lsnp2 = ACCESS_ONCE(head->first);
19    do {
20        lsnp1 = lsnp2;
21        lsp->next = lsnp1;
22        lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23    } while (lsnp1 != lsnp2);
24    rcu_read_unlock();
25    return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30     struct liststack *lsp;
31
32     lsp = container_of(rhp, struct liststack, rh);
33     free(lsp);
34 }
35
36 void *ls_pop(struct liststackhead *head)
37 {
38     struct liststack *lsp;
39     struct liststack *lsnp1;
40     struct liststack *lsnp2;
41     void *data;
42
43     rcu_read_lock();
44     lsnp2 = rcu_dereference(head->first);
45     do {
46         lsnp1 = lsnp2;
47         if (lsnp1 == NULL) {
48             rcu_read_unlock();
49             return NULL;
50         }
51         lsp = rcu_dereference(lsnp1->next);
52         lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
53     } while (lsnp1 != lsnp2);
54     data = rcu_dereference(lsnp2->t);
55     rcu_read_unlock();
56     call_rcu(&lsnp2->rh, ls_rcu_free_cb);
57     return data;
58 }

```

Figure 21: List-Based-Stack Whole-Program Approach, 2 of 2

It will also be necessary to check whether Linux-kernel usage expects dependency chains to pass through globals and heap objects that are in some way thread-local. If there are such use cases, and if they are sane and cannot easily be changed to use local variables, should `[[carries_dependency]]` be used to flag dependency-carrying globals and heap objects?

This approach has the following advantages and disadvantages:

1. This approach requires that the C language add the `[[carries_dependency]]` attribute if dependency chains are to span multiple translation units, as is the case in some parts of the Linux kernel.
2. The implementation is likely to be somewhat simpler because only those dependency chains passing through local variables, compiler-generated temporaries, compiler-visible function arguments, and compiler-visible return values need be traced. One could also argue that function arguments and return values marked with `[[carries_dependency]]` attribute also need to be traced.
3. Many irrelevant dependency chains are pruned by default, thus fewer `std::kill_dependency()` calls are required.
4. Although optimizations on dependency chains must be restricted, the restricted scope of dependency chains reduces the impact of these restrictions.
5. Applying this approach to the Linux kernel would only require the addition of markings on function parameters and return values corresponding to cross-translation-unit function calls. However, there are a significant number of these, so this approach can expect significant resistance from the Linux community.

It is expected that modeling this approach should be no more difficult than for the current C11 and C++11 standards.

This approach allows local-variable annotations to be dropped, as shown in Figure 22 and 23

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack *first;
6 };
7
8 struct liststack {
9   struct liststack *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 _Carries_dependency
15 void *ls_front(struct liststackhead *head)
16 {
17   void *data;
18   struct liststack *lsp;
19
20   rcu_read_lock();
21   lsp = rcu_dereference(head->first);
22   if (lsp == NULL)
23     data = NULL;
24   else
25     data = rcu_dereference(lsp->t);
26   rcu_read_unlock();
27   return data;
28 }

```

Figure 22: List-Based-Stack Local-Variable Restriction, 1 of 2

7.6 Mark Dependency-Carrying Local Variables

This approach, suggested offlist by Clark Nelson, uses the `[[carries_dependency]]` attribute to mark non-static local-scope variables as carrying a dependency, in addition to its current use marking function arguments and return values as carrying dependencies. It is not permissible to mark global variables or structure members with this attribute. Assigning from a `[[carries_dependency]]` object to a non-`[[carries_dependency]]` object results in an implicit `std::kill_dependency()`.

This approach is similar to that of Section 7.3, except that it uses an attribute rather than a type modifier. As such, it has many of the advantages and disadvantages of that approach, however, some believe that an attribute-based approach will be more acceptable to the committee than would a type-modifier approach.¹³ However, this approach does require

¹³ Lawrence Crowl suggests a third approach, namely a variable modifier.

```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3   struct liststack *lsp;
4   struct liststack *lsnp1;
5   struct liststack *lsnp2;
6   size_t sz;
7
8   sz = sizeof(*lsp);
9   sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10  sz *= CACHE_LINE_SIZE;
11  lsp = malloc(sz);
12  if (!lsp)
13    return -ENOMEM;
14  if (!t)
15    abort();
16  lsp->t = t;
17  rcu_read_lock();
18  lsnp2 = ACCESS_ONCE(head->first);
19  do {
20    lsnp1 = lsnp2;
21    lsp->next = lsnp1;
22    lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23  } while (lsnp1 != lsnp2);
24  rcu_read_unlock();
25  return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rh)
29 {
30   struct liststack *lsp;
31
32   lsp = container_of(rh, struct liststack, rh);
33   free(lsp);
34 }
35
36 _Carries_dependency
37 void *ls_pop(struct liststackhead *head)
38 {
39   struct liststack *lsp;
40   struct liststack *lsnp1;
41   struct liststack *lsnp2;
42   void *data;
43
44   rcu_read_lock();
45   lsnp2 = rcu_dereference(head->first);
46   do {
47     lsnp1 = lsnp2;
48     if (lsnp1 == NULL) {
49       rcu_read_unlock();
50       return NULL;
51     }
52     lsp = rcu_dereference(lsnp1->next);
53     lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54   } while (lsnp1 != lsnp2);
55   data = rcu_dereference(lsnp2->t);
56   rcu_read_unlock();
57   call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58   return data;
59 }

```

Figure 23: List-Based-Stack Local-Variable Restriction, 2 of 2

that C add attributes.

This leave the question of which operators transmit dependency chains from one `[[carries_dependency]]` object to another. Section 3.2 indicates that the following operators should transmit dependency status from one local variable or temporary to another: `->`, infix `=`, casts, prefix `&`, prefix `*`, `[]`, infix `+`, infix `-`, ternary `?:`, infix (bitwise) `&`, and probably also `|`. Similarly, Section 3.3 shows that the following operators should imply a `std::kill_dependency()`: `()`, `!`, `==`, `!=`, `&&`, `||`, infix `*`, `/`, and `%`.

This approach has the following advantages and disadvantages:

1. This approach requires that the C language add the `[[carries_dependency]]` attribute.
2. The implementation is likely to be simpler because only those dependency chains passing through variables marked with the `[[carries_dependency]]` attribute need be traced.
3. Many irrelevant dependency chains are pruned by default, thus fewer `std::kill_dependency()` calls are required.
4. The `[[carries_dependency]]` calls serve as valuable documentation of the developer's intent.
5. Although optimizations on dependency chains must be restricted, use of explicit `[[carries_dependency]]` greatly reduces unnecessary restriction of optimizations on unintentional dependency chains.
6. Applying this to the Linux kernel would require significant marking of variables carrying dependencies, given that the Linux kernel currently requires no such markings.
7. Common types of abstraction need to be handled correctly. For example, there are more than 500 invocations of `list_for_each_entry_rcu()` in the v4.1 Linux kernel, each of which heads a distinct dependency chain. In addition, some

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack *first;
6 };
7
8 struct liststack {
9   struct liststack *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 _Carries_dependency
15 void *ls_front(struct liststackhead *head)
16 {
17   _Carries_dependency void *data;
18   _Carries_dependency struct liststack *lsp;
19
20   rcu_read_lock();
21   lsp = rcu_dereference(head->first);
22   if (lsp == NULL)
23     data = NULL;
24   else
25     data = rcu_dereference(lsp->t);
26   rcu_read_unlock();
27   return data;
28 }

```

Figure 24: List-Based-Stack Marked Local Variables, 1 of 2

of these distinct dependency chains invoke common functions and macros. It is not clear that compile-time marking suffices for these cases.

It is expected that modeling this approach should be no more difficult than for the current C11 and C++11 standards.

This approach results in code as shown in Figures 24 and 25, where the `[[carries_dependency]]` attributes have been replaced with a mythical `_Carries_dependency` C keyword.

7.7 Explicitly Tail-Marked Dependency Chains

This approach, suggested at Rapperswil by Olivier Giroux, can be thought of as the inverse of `std::kill_dependency()`. Instead of explicitly marking where the dependency chains terminate, Olivier's proposal uses a `std::dependency()` primitive to indicate the locations in the code that the dependency chains are required to reach. The first ar-


```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3     struct liststack *lsp;
4     struct liststack *lsnp1;
5     struct liststack *lsnp2;
6     size_t sz;
7
8     sz = sizeof(*lsp);
9     sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10    sz *= CACHE_LINE_SIZE;
11    lsp = malloc(sz);
12    if (!lsp)
13        return -ENOMEM;
14    if (!t)
15        abort();
16    lsp->t = t;
17    rcu_read_lock();
18    lsnp2 = ACCESS_ONCE(head->first);
19    do {
20        lsnp1 = lsnp2;
21        lsp->next = lsnp1;
22        lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23    } while (lsnp1 != lsnp2);
24    rcu_read_unlock();
25    return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30     struct liststack *lsp;
31
32     lsp = container_of(rhp, struct liststack, rh);
33     free(lsp);
34 }
35
36 _Carries_dependency
37 void *ls_pop(struct liststackhead *head)
38 {
39     _Carries_dependency struct liststack *lsp;
40     struct liststack *lsnp1;
41     _Carries_dependency struct liststack *lsnp2;
42     _Carries_dependency void *data;
43
44     rcu_read_lock();
45     lsnp2 = rcu_dereference(head->first);
46     do {
47         lsnp1 = lsnp2;
48         if (lsnp1 == NULL) {
49             rcu_read_unlock();
50             return NULL;
51         }
52         lsp = rcu_dereference(lsnp1->next);
53         lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54     } while (lsnp1 != lsnp2);
55     data = rcu_dereference(lsnp2->t);
56     rcu_read_unlock();
57     call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58     return data;
59 }

```

Figure 25: List-Based-Stack Marked Local Variables,
2 of 2

```

1 p = atomic_load_explicit(&gp, memory_order_consume);
2 if (p != NULL)
3     do_it(atomic_dependency(p, gp));

```

Figure 26: Explicit Dependency Operations

```

1 void foo(struct bar *q [[carries_dependency]])
2 {
3     if (q != NULL)
4         do_it(atomic_dependency(q->b, q));
5 }
6
7 p = atomic_load_explicit(&gp, memory_order_consume);
8 foo(atomic_dependency(p, gp));

```

Figure 27: Explicit Dependency Operations and carries_dependency

gument to `std::dependency()` is the value to which the dependency must be carried, and the second argument is the variable that heads the dependency chain, in other words, the second argument is the variable that was loaded from by a `memory_order_consume` load. This proposal differs from the others in that it is expected to be implemented not necessarily by preserving the dependency, but instead by inserting barriers in those cases where optimizations have eliminated any required dependencies. The goal here is to impose minimal restrictions on optimizations of code containing dependency chains.

A C-language example is shown in Figure 26, where `std::dependency()` is transliterated to the C-language `atomic_dependency()` function. On line 3, `atomic_dependency()` returns the value of its first argument (`p`), while ensuring that the data dependency from the `memory_order_consume` load from `gp` is faithfully reflected in the assembly language implementing this code fragment. The assembly-language reflection of this dependency might be in terms of an assembly-language dependency (for example, on ARM or PowerPC), implicit memory ordering (for example, on x86 or mainframe), or by an explicit memory-barrier instruction. However, if there was no `atomic_dependency()` function, the compiler would be under no obligation to preserve the dependency.¹⁴

¹⁴ Would it be better to have the second argument to `atomic_dependency()` be a label rather than an expression?

These explicitly specified dependencies may be combined with `[[carries_dependency]]` attributes on function arguments, for example, as shown in Figure 27. Note the interplay of `atomic_dependency()` and `[[carries_dependency]]`, where line 8 establishes the dependency between the load from `gp` and the `[[carries_dependency]]` argument `q` of `foo()`, and where line 4 establishes the further dependency between argument `q` of `foo()` and `do_it()`'s argument.

This approach is not yet complete. One issue is the possibility of a given operation being dependent on multiple `memory_order_consume` loads. One approach is of course to omit this functionality, and another is to allow `atomic_dependency()` to allow an expression as its first argument and a variable list of `memory_order_consume` loaded variables.

Another issue is connecting `[[carries_dependency]]` return values to subsequent `atomic_dependency()` invocations. There are a number of possible resolutions to this issue. One approach would be to use `[[carries_dependency]]` attribute to mark the declaration of the variable to which the function's return value is assigned, bringing the proposal from Section 7.6 to bear. In the special case where the `memory_order_consume` load is in the same function body as the `atomic_dependency()` that depends on it, the `atomic_dependency()` could reference the variable that was the source of the original `memory_order_consume` load. Another approach would be to allow function-return `carries_dependency` attributes to define names that could be used by later `atomic_dependency()` invocations.

A third issue arises when `atomic_dependency()` must be applied after the head of the dependency chain has gone out of scope, for example, if the head was contained in a variable defined in an inner scope that has since been exited.

A fourth issue arises if optimizations along a needed dependency chain allow ordering the dependent operation to precede the head of the dependency chain, in which case inserting barriers would be ineffective. The current proposal for addressing this issue is to suppress memory-movement optimizations across the `atomic_dependency()`, perhaps us-

ing something like `atomic_signal_fence()` or the Linux kernel's `barrier()` macro. This approach allows dependency checking and fence insertion to be carried out as a final pass in the compilation process.

This approach has the following advantages and disadvantages:

1. This approach requires that the C language add the `[[carries_dependency]]` attribute.
2. The implementation is likely to be simpler because only those dependency chains having explicit `atomic_dependency()` calls (and, optionally, intermediate `[[carries_dependency]]` attributes) need be traced.
3. Irrelevant dependency chains are pruned by default, with no `std::kill_dependency()` calls required.
4. The `atomic_dependency()` calls serve as valuable documentation of the developer's intent.
5. Although optimizations on dependency chains must be restricted, use of explicit `atomic_dependency()` greatly reduces unnecessary restriction of optimizations on unintentional dependency chains.
6. Applying this to the Linux kernel would require significant marking of dependency chains, given that the Linux kernel currently relies on implicit ends of dependency chains.
7. Common types of abstraction need to be handled correctly. For example, there are more than 500 invocations of `list_for_each_entry_rcu()` in the v4.1 Linux kernel, each of which heads a distinct dependency chain. In addition, some of these distinct dependency chains invoke common functions and macros. It is not clear that compile-time marking suffices for these cases.

It is not yet known whether this approach can be reasonably modeled.

The result is shown in Figures 28 and 29.

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack *first;
6 };
7
8 struct liststack {
9   struct liststack *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 _Carries_dependency
15 void *ls_front(struct liststackhead *head)
16 {
17   void *data;
18   struct liststack *lsp;
19
20   rcu_read_lock();
21   lsp = rcu_dereference(head->first);
22   if (lsp == NULL)
23     data = NULL;
24   else
25     data =
26       rcu_dereference(atomic_dependency(lsp->t,
27                                         head->first));
28   rcu_read_unlock();
29   return atomic_dependency(data, lsp->t);
30 }

```

Figure 28: List-Based-Stack Tail-Marked Dependencies, 1 of 2

```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3   struct liststack *lsp;
4   struct liststack *lsnp1;
5   struct liststack *lsnp2;
6   size_t sz;
7
8   sz = sizeof(*lsp);
9   sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10  sz *= CACHE_LINE_SIZE;
11  lsp = malloc(sz);
12  if (!lsp)
13    return -ENOMEM;
14  if (!t)
15    abort();
16  lsp->t = t;
17  rcu_read_lock();
18  lsnp2 = ACCESS_ONCE(head->first);
19  do {
20    lsnp1 = lsnp2;
21    lsp->next = lsnp1;
22    lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23  } while (lsnp1 != lsnp2);
24  rcu_read_unlock();
25  return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30   struct liststack *lsp;
31
32   lsp = container_of(rhp, struct liststack, rh);
33   free(lsp);
34 }
35
36 _Carries_dependency
37 void *ls_pop(struct liststackhead *head)
38 {
39   struct liststack *lsp;
40   struct liststack *lsnp1;
41   struct liststack *lsnp2;
42   void *data;
43
44   rcu_read_lock();
45   lsnp2 = rcu_dereference(head->first);
46   do {
47     lsnp1 = lsnp2;
48     if (lsnp1 == NULL) {
49       rcu_read_unlock();
50       return NULL;
51     }
52     lsp = rcu_dereference(lsnp1->next);
53     lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54   } while (lsnp1 != lsnp2);
55   data = rcu_dereference(atomic_dependency(lsnp2->t, lsnp2));
56   rcu_read_unlock();
57   call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58   return atomic_dereference(data, lsnp2->t);
59 }

```

Figure 29: List-Based-Stack Tail-Marked Dependencies, 2 of 2

```

1 struct foo {
2     struct foo *a;
3     struct foo *b;
4     struct foo *c;
5     int d;
6 };
7
8 p = atomic_load_explicit(&gp, memory_order_consume,
9                         p->a, p->b);
10 qa = p->a; /* Dependency carried. */
11 qb = p->b; /* Dependency carried. */
12 qc = p->c; /* No dependency carried. */
13 d = p->d; /* No dependency carried. */

```

Figure 30: Explicit Dependency Operations and Augmented Load

7.8 Explicitly Head-Marked Dependency Chains

This approach, suggested via email by Olivier Giroux, can be thought of as another inverse of `std::kill_dependency()`. In this case the heads of the dependency chains are marked, indicating to which pointed-to objects dependencies should be carried. This description is an extrapolation of a very concise proposal, and corrections are welcome.

The general idea is to provide an augmented form of the `load()` member function that indicates dependencies, for example, `x.load(memory_order_consume, x->next)` would cause a dependency to be carried through the `->next` field, but through no other field. This is shown in Figure 30, where the explicit dependency information on line 9 causes lines 10 and 11 to carry a dependency, but lines 12 and 13 not to do so.

Some open questions regarding this approach:

1. How does this interact with arguments and return values? Do the corresponding annotations need to indicate to which fields dependencies might be carried? Should mismatches be considered an error, and if so, which sorts of mismatches?
2. How are opaque types handled? For example, consider the Linux kernel linked-list facility, which embeds a `list_head` structure into the enclosing object that is to be placed on the list. The `memory_order_consume` load returns

a `(struct list_head *)`, but it may be necessary to carry a dependency to one or more of the fields in the enclosing object. Should this be handled via something like `x.load(memory_order_consume, *x)`, but if so, doesn't this re-introduce the need for lots of `std::kill_dependency()` calls?

3. Larger structures might have quite a few fields that need dependencies carried. Should there be some sort of shorthand to make this easier to code, for example, tagging the fields needing dependency ordering in the declaration of the `struct` or `class`?
4. Common types of abstraction need to be handled correctly. For example, there are more than 500 invocations of `list_for_each_entry_rcu()` in the v4.1 Linux kernel, each of which heads a distinct dependency chain. In addition, some of these distinct dependency chains invoke common functions and macros. It is not clear that compile-time marking suffices for these cases.

7.9 Restricted Dependency Chains

This approach restricts dependency chains to operations for which compilers would naturally carry dependencies. As such, this approach can be considered to be a refinement of the whole-program option (Section 7.4), restricted as described in Section 4.1.2, but also omitting control dependencies and RCU-protected array indexes. It can also be thought of as a refinement of the local-variable restriction (Section 7.5). As always, a `memory_order_consume` load heads a dependency chain, and as always, a `memory_order_consume` load may be implemented with a simple load instruction on a wide variety of architectures, including x86, ARM, and Power.

This results in a specific list of operations that extend dependency chains (Section 7.9.1) and a separate specific list of operations that terminate such chains (Section 7.9.2). Section 7.9.3 looks at how this approach relates to the Linux kernel, and Section 7.9.4 lists this approach's advantages and disadvantages.

```

1 #define rcu_dereference(x) \
2   atomic_load_explicit((x), memory_order_consume);
3
4 struct liststackhead {
5   struct liststack *first;
6 };
7
8 struct liststack {
9   struct liststack *next;
10  void *t;
11  struct rcu_head rh;
12 };
13
14 _Carries_dependency
15 void *ls_front(struct liststackhead *head)
16 {
17   void *data;
18   struct liststack *lsp;
19
20   rcu_read_lock();
21   lsp = rcu_dereference(head->first, head->first->t);
22   if (lsp == NULL)
23     data = NULL;
24   else
25     data =
26       rcu_dereference(lsp->t, *lsp->t)); /* ??? */
27   rcu_read_unlock();
28   return data;
29 }
30 }

```

Figure 31: List-Based-Stack Head-Marked Dependencies, 1 of 2

It is worth repeating that key goals include minimizing the need for `std::kill_dependency()` and minimizing unsolicited memory-barrier instructions.

7.9.1 Extending Dependency Chains

The following categories of primitive operations extend dependency chains:¹⁵

1. Moving, copying, and casting.
2. Pointer offsets.
3. Dereferencing and address-of, including class-member access.
4. Integer operations.
5. Miscellaneous operators.

¹⁵ In case of operator overloading, the actual functions called must be analyzed in order to determine their effects on dependency chains.

```

1 int ls_push(struct liststackhead *head, void *t)
2 {
3   struct liststack *lsp;
4   struct liststack *lsnp1;
5   struct liststack *lsnp2;
6   size_t sz;
7
8   sz = sizeof(*lsp);
9   sz = (sz + CACHE_LINE_SIZE - 1) / CACHE_LINE_SIZE;
10  sz *= CACHE_LINE_SIZE;
11  lsp = malloc(sz);
12  if (!lsp)
13    return -ENOMEM;
14  if (!t)
15    abort();
16  lsp->t = t;
17  rcu_read_lock();
18  lsnp2 = ACCESS_ONCE(head->first);
19  do {
20    lsnp1 = lsnp2;
21    lsp->next = lsnp1;
22    lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
23  } while (lsnp1 != lsnp2);
24  rcu_read_unlock();
25  return 0;
26 }
27
28 static void ls_rcu_free_cb(struct rcu_head *rhp)
29 {
30   struct liststack *lsp;
31
32   lsp = container_of(rhp, struct liststack, rh);
33   free(lsp);
34 }
35
36 _Carries_dependency
37 void *ls_pop(struct liststackhead *head)
38 {
39   struct liststack *lsp;
40   struct liststack *lsnp1;
41   struct liststack *lsnp2;
42   void *data;
43
44   rcu_read_lock();
45   lsnp2 = rcu_dereference(head->first, head->first->next);
46   do {
47     lsnp1 = lsnp2;
48     if (lsnp1 == NULL) {
49       rcu_read_unlock();
50       return NULL;
51     }
52     lsp = rcu_dereference(lsnp1->next, lsnp1->next->t);
53     lsnp2 = cmpxchg(&head->first, lsnp1, lsp);
54   } while (lsnp1 != lsnp2);
55   data = rcu_dereference(lsnp2->t, *lsnp2->t));
56   rcu_read_unlock();
57   call_rcu(&lsnp2->rh, ls_rcu_free_cb);
58   return data;
59 }

```

Figure 32: List-Based-Stack Head-Marked Dependencies, 2 of 2

```

1 #define rcu_ptr_extract(p) \
2 ({ \
3     uintptr_t ___ip = (uintptr_t)(p); \
4     \
5     ___ip = ___ip & ~0x7; \
6     (typeof(p)) ___ip; \
7 })
8
9 #define rcu_ptr_set_bits(p, bits) \
10 ({ \
11     uintptr_t ___ip = (uintptr_t)(p); \
12     \
13     ___ip = ___ip & ~0x7; \
14     ___ip = ___ip | (bits); \
15     (typeof(p)) ___ip; \
16 })
17
18 #define rcu_ptr_get_bits(p, bits) \
19 ({ \
20     uintptr_t ___ip = (uintptr_t)(p); \
21     \
22     ___ip & ~0x7; \
23 })

```

Figure 33: Pointers and Bit Manipulation on 64-Bit System

```

1 p = rcu_dereference(gp);
2 *p = 5;

```

Figure 34: Extending Dependency Chain on Left-Hand Side

Any other operation terminates a dependency chain. The operations that extend dependency chains are covered in more detail below.

Moving, Copying, and Casting: Values that are part of a dependency chain may be moved, copied, and casted (in some cases), and the dependency chain will propagate to the result.

1. If any value is part of a dependency chain, then using that value as the left-hand side of an assignment expression extends the chain to cover the assignment. This rule is exercised in the Linux kernel by stores into fields making up an RCU protected data element. This is illustrated by Figure 34.
2. If any value is part of a dependency chain, then using that value as the right-hand side of an assignment expression extends the chain to cover both the assignment and the value returned by that assignment statement. Line 20 of Figure 20 shows how this rule may be used to extend a dependency chain into a local variable.
3. If any value that is part of a dependency chain is stored to a non-shared variable, then any value loaded by a later load from that same variable by that same thread is also part of the dependency chain. Lines 20 and 24 of Figure 20 illustrate this rule, though this rule would apply even if local variable `lsp` was a `intptr_t` instead of a pointer. Note that the job of determining whether or not a given variable is non-shared falls to the developer, *not* the compiler. That said, a high-QoI compiler might choose to make this determination in order to issue helpful diagnostic messages.
4. If a pointer that is part of a dependency chain is stored to any variable, then any value loaded by a later load from that same variable by that same thread is also part of the dependency chain. Lines 20 and 24 of Figure 20 illustrate this rule, though this rule would apply even if local variable `lsp` was instead a shared variable. As before, determining whether or not the store and

load were carried out by the same thread falls to the developer, not to the compiler.

5. If a pointer is part of a dependency chain, then casting it (either explicitly or implicitly) to any pointer-sized type extends the chain to the result. Line 3 of Figure 33 illustrates this rule.
6. If a value of type `intptr_t` or `uintptr_t` is part of a dependency chain, then casting it to a pointer type extends the chain to the result. Line 6 of Figure 33 illustrates this rule. Note that many implementations do not differentiate between `intptr_t` and `uintptr_t` on the one hand and other integral types on the other. Such implementations would likely loosen this restriction to cover a larger subset of the integral types.
7. If a pointer is part of a dependency chain, then if that pointer is used as the argument of a function call, the dependency chain extends to the corresponding parameter.
8. If a function returns a pointer that is part of a dependency chain, the dependency chain extends to the returned value in the calling function.

Pointer Offsets: If a given pointer is part of a dependency chain, then integral offsets to that pointer are also part of that dependency chain.

1. If a pointer is part of a dependency chain, then adding an integral value to that pointer extends the chain to the resulting value. This applies for both positive and negative integers, and also to addition via the infix `+` operator and via the postfix `[]` operator. Note that the addition must be carried out on a pointer: Casting to an integral type and then carrying out the addition will break the dependency chain. Therefore, instead of casting to an integral type to carry out the addition, cast to a pointer to `char`.¹⁶ Line 24 of

¹⁶ Yes, some old systems had strange formats for character pointers, and this restriction does exclude those systems from this nuance of dependency ordering. However, to the best of my knowledge, all such systems were uniprocessors, so this is not a real problem.

Figure 20 illustrates this, given that the `->t` acts as a pointer offset prior to indirection.

2. If a pointer is part of a dependency chain, then subtracting an integer from that pointer extends the chain to the resulting value. This applies for both positive and negative integers. Again, casting to an integral type and then carrying out the subtraction will break the dependency chain, so instead cast to a pointer to `char`. The Linux-kernel `container_of()` macro illustrates this. This macro is used to find the beginning of a structure given a pointer to a field within that same structure.
3. Note that class-member access operators (`.` and `->`) can be thought of as computing an offset as part of their execution.

Dereferencing and Address-Of: Dereferencing a pointer that is part of a dependency chain extends the dependency chain to the result, but only when the resulting value is a pointer type. Taking the address of a pointer that is part of a dependency chain, and then dereferencing the resulting pointer, extends the dependency chain to the result.

1. If a pointer is part of a dependency chain, then dereferencing it using the prefix `*` operator extends the chain through the dereference operation. Line 24 of Figure 20 illustrates this, given that the `->t` acts as a pointer offset prior to indirection.
2. If a pointer is part of a dependency chain, then dereferencing it using the `->` field-selection operator extends the chain to the field. Note that the when the `->` operator is followed by one or more `.` operators, these latter operators are equivalent to adding a constant integer to the original pointer. Line 24 of Figure 20 directly illustrates this rule.
3. If a pointer is part of a dependency chain, then applying the unary `&` address-of operator, optionally casting this address to a pointer type (perhaps repeatedly to different pointer types,

either explicitly or implicitly), then applying the `*` dereference operator extends the chain to the result. This is used by some of the Linux-kernel list-processing macros.

Integer Operations: It is important to note that performing integer operations on pointer values and then attempting to use the result as a pointer is outside of the current standard. Nevertheless, there are a great many use cases in this area, so the following rules apply. Please note that these rules cannot be part of the standard in its current form, but instead must be negotiated with individual implementers.

1. If a value of type `intptr_t` or `uintptr_t` is part of a dependency chain, then the bitwise infix `&` and `|` operators extend the dependency chain to the resulting value. Line 5 of Figure 33 illustrates this rule.
2. If a value of type `intptr_t` or `uintptr_t` is part of a dependency chain, the the bitwise infix `^` operator extends the dependency chain to the resulting value. The use case for this traversal of buddy-allocator-like lists or dense-array heaps, but it is not clear whether these use cases justify this addition to dependency ordering.

Miscellaneous Operations: The following operations also extend dependency chains.

1. If a pointer is part of a dependency chain, and that pointer appears in the operand of a `?:` operator selected by the condition, then the chain extends to the result. Please note that `?:` does not extend chains from its condition, only from its second or third argument.
2. If a pointer is part of a dependency chain, and that pointer appears in the right-hand operand of a `,` operator, then the chain extends to the result. Please note that the `,` operator does not extend chains from its left-hand operand, only from its right-hand operand.
3. If a pointer to a function is part of a dependency chain, then invoking the pointed-to function extends the chain from the pointer to the

```

1 struct bar {
2     struct bar *next;
3     int a;
4     int b;
5 };
6 struct bar *head = { &head, 1, 2 };
7
8 for (p = head->next; p; p = rcu_dereference(p->next)) {
9     foo += p->a;
10    if (p == &head)
11        break;
12 }
13 bar *= head->b;
```

Figure 35: Back-Propagation of Dependency-Chain Breakage Due to Comparisons

```

1 if (p > &foo)
2     do_something(p);
3 else if (p < &foo)
4     do_something_else(p);
5 else
6     do_something_nodep(p);
```

Figure 36: Inequality-Comparison Dependency-Chain Breakage

instructions executed. Note that the exact mechanism used to update instructions is implementation defined, and might require use of special instruction-cache-flush operations.¹⁷

4. If a given operation extends a dependency chain, then so does its atomic counterpart. For example, the rules applying to assignments also apply to atomic loads and stores. It also applies to atomic exchange and and atomic compare and swap.

7.9.2 Terminating Dependency Chains

Even though all other operations terminate dependency chains, there are a few that deserve special mention:

1. Equality comparisons.

¹⁷ This may sound strange, but just you try implementing dynamic linking without the ability to update instructions! That said, this is very clearly outside of the current standard, so this item is something to be negotiated with individual implementers.

2. Narrowing magnitude comparisons.
3. Narrowing arithmetic operations.
4. Narrowing bitwise operations.
5. Storing non-pointers into shared variables.
6. Passing values between threads without using a `memory_order_consume` load.
7. Undefined behavior.
8. Use of `std::kill_dependency`.

Each of these is covered below.

Equality comparisons: If a pointer is part of a dependency chain, then a `==` or `!=` comparison that compares equal to some other pointer, where that other pointer is not part of any dependency chain, will cause any uses of the original pointer to no longer be part of the dependency chain. This dependency-chain breakage can back-propagate to earlier uses of the pointer, so that in Figure 35, if the comparison on line 10 compares equal, then the access on line 9 is not part of the dependency chain. This is admittedly a rather strange code fragment, and besides, the Linux-kernel `barrier()` macro could prevent this if placed between lines 9 and 10. Furthermore, the Linux kernel's list macros avoid this situation because the equal comparison terminates the loop.

So what if the compiler introduces an equality comparison? This might happen when doing feedback-directed optimization, where the compiler might notice (for example) that a particularly statically allocated structure was almost always the first element on a given list. The compiler might therefore introduce a specialization optimization, comparing the addresses and generating code using the statically allocated structure on equals comparison. On the one hand, in the cases where the Linux kernel adds a statically allocated structure to an RCU-protected linked data structure, that structure has been initialized at compile time, so that dependency ordering is not required. On the other hand, this appears to be an extremely dubious optimization for linked data structures: In a great many cases, the added overhead of

the comparison would overwhelm the benefits of generating code based on the statically allocated structure.

High-quality implementations would therefore be expected to provide means for disabling this sort of optimization, especially for pointers obtained from the heap. After all, use of statically allocated structures in RCU-protected lists could be quite useful during out-of-memory conditions, in which case the specialization optimization would almost always reduce performance, which is not what optimizations are supposed to be doing.

Alternatively, an intrinsic could provide comparison, but avoid breaking dependency chains. For example, a `T std::pointer_cmp_eq_dep(T *pd, T* p)` intrinsic could compare the two pointers, but preserve dependencies carried by `pd` even if they compare equal. For completeness, a `T std::pointer_cmp_ne_dep(T *pd, T* p)` intrinsic could also be provided for not-equal comparisons. Note that there is no way to compare two pointers carrying dependencies and preserve the dependency for both. This is because there are currently no use cases requiring this, and requiring it would require the compiler to be less efficient in its register usage.

Narrowing magnitude comparisons: A series of `>`, `<`, `>=`, or `<=` operators that informs the compiler of the exact value of a pointer causes that pointer to no longer be part of the dependency chain. See Figure 36 for an example of this. On line 6 of this figure, the compiler knows that the value of `p` is equal to `&foo`, so although there is dependency ordering to lines 2 and 4, there is no dependency ordering to line 6. This dependency-chain breakage can back-propagate, just as for equality comparisons. However, dependencies are maintained for normal uses, for example, the use of comparisons for deadlock avoidance when acquiring locks contained in multiple RCU-protected data elements.

As with equality comparison, an intrinsic could provide comparison, but avoid breaking dependency chains. For example, `T std::pointer_cmp_gt_dep(T *pd, T* p)`, `T std::pointer_cmp_ge`

`dep(T *pd, T* p)`, `T std::pointer_cmp_lt_dep(T *pd, T* p)`, and `T std::pointer_cmp_le_dep(T *pd, T* p)` intrinsics could compare the two pointers, but preserve dependencies carried by `pd` even if a series of comparisons allowed the compiler to deduce the exact value of the pointer.

Narrowing arithmetic operations: If a pointer is part of a dependency chain, and if the values added to or subtracted from that pointer cancel the pointer value so as to allow the compiler to precisely determine the resulting value, then the resulting value will not be part of any dependency chain. For example, if `p` is part of a dependency chain, then `((char *)p-(uintptr_t)p)+65536` will not be.¹⁸

Narrowing bitwise operations: If a value of type `intptr_t` or `uintptr_t` is part of a dependency chain, and if that value is one of the operands to an `&` or `|` infix operator whose result has too few or too many bits set, then the resulting value will not be part of any dependency chain. For example, on a 64-bit system, if `p` is part of a dependency chain, then `(p & 0x7)` provides just the tag bits, and normally cannot even be legally dereferenced. Similarly, `(p | ~0)` normally cannot be legally dereferenced. However, `(p & ~0x7)` will provide a usable pointer that is part of `p`'s dependency chain. Setting or clearing bits that are not used by the implementation or that would be zero for a properly aligned object will not break the dependency chain. That said, the compiler might not know the definition of “properly aligned”, for example, in the cases of manually cache-aligned or page-aligned objects.

Note that the `^` exclusive-OR operator can also break dependency chains, most straightforwardly via something like `a ^ a`.

Storing non-pointers into shared variables: If a non-pointer value that is part of a dependency chain is stored into a shared variable, then the dependency

chain does not extend to a later load from that variable.

Passing values between threads: If a value that is part of a dependency chain is stored into a variable by one thread, and loaded from that same variable by some other thread using either a non-atomic load or a `memory_order_relaxed` load, then the dependency chain does not extend to the second thread. To get this effect, the second thread would instead need to use a `memory_order_consume` load. Note that this would extend the dependency chain even if the corresponding store was a `memory_order_relaxed` store because the required store-side ordering is provided by the dependency chain.

Undefined behavior: If undefined behavior is invoked, then, consistent with the notion of undefined behavior, there are no dependency-chain guarantees.

If a given pointer, `intptr_t`, or `uintptr_t` is used such that only one value avoids undefined behavior, then the dependency chain is broken in the same way as it would be in the case of an equality comparison with that same value.

kill_dependency(): The result of calling `std::kill_dependency` is never part of any dependency chain. This operation can be used to suppress diagnostics that implementations might omit for likely misuses of dependency ordering.

7.9.3 Restricted Dependency Chains and the Linux Kernel

This covers all known pointer-based RCU uses in the Linux kernel, aside from RCU-protected array indexes (more on these later). However, as noted earlier, these restrictions might prove too constraining for future code. Therefore, it might be necessary to combine this approach with some variation on one of the methods for explicitly marking variables, formal parameters, and return values that are intended to carry dependencies.

Section 3.2 discussed dependency chains headed by `memory_order_consume` loads of integers that are

¹⁸ That said, 5.7p4 of C++ and 6.5.6p8 of C both say that indexing outside of an object is undefined behavior, so the loss of dependency ordering is likely the least of the problems here.

```

1 p = atomic_load_explicit(gp, memory_order_consume);
2 if (p == ptr_a) {
3   q = std::kill_dependency(p);
4   a = q->special_a;
5 } else {
6   a = p->normal_a;
7 }

```

Figure 37: Avoiding Diagnostic Due To Dependency-Ordering Value-Narrowing Hazard

later used as array indexes or pointer offsets. Although there are a (very) few such uses in the Linux kernel, accommodating dependency chains headed by loads of integers greatly complicates the handling of dependency chains.¹⁹ For example, given an integer `x` produced by a `memory_order_consume` load, we must correctly handle expressions containing `(x - x)`, including cases where the cancellation is not at all obvious from the source code. In contrast, given a pointer `p`, the expression `(p - p)` not a pointer, and the rules given above do not require carrying a dependency through such an expression. This approach therefore excludes dependency chains headed by `memory_order_consume` loads from non-pointer atomic variables.

This raises the question of what should be done about the Linux kernel code that relies on ordering carried through integer array indexes. Paul has answered this question by creating a Linux-kernel patch that removes the kernel’s dependency on RCU-protected array indexes [22], and this patch was accepted into the v4.2 release of the Linux kernel.

7.9.4 Restricted Dependency Chains: Advantages and Disadvantages

This approach to dependency ordering has the following advantages and disadvantages:

1. There is no need for compilers to trace dependency chains. Instead, dependencies are an automatic result of current code-generation and optimization practices.

¹⁹ And for this reason, carrying of dependencies via integers is no longer supported in the Linux kernel as of v4.2, except in a few very restricted instances.

2. There would be little or no limitation on compiler optimizations. Compilers are no longer required to establish artificial dependencies for expressions such as `(x - x)` or to detect value-narrowing hazards involving `==` and `!=`.
3. The breaking of dependency chains when a pointer compares equal to some other pointer might prove to be onerous, but it is not a problem for current Linux use cases.²⁰ The most common cases are comparison against a list header (in which case an equality comparison terminates the traversal) and comparison against NULL (in which case an equality comparison indicates a pointer that cannot be dereferenced in any case).
4. The compiler is prohibited from carrying out value-speculation optimizations on pointers or values of type `intptr_t` or `uintptr_t` that have been cast from a pointer type and subjected to no operations other than infix `&`, `|`, and `^`. (Is this an advantage or a disadvantage? The answer to this question is left to the reader.)
5. In a great many cases, dependency chains can reliably pass through library functions compiled by pre-C11 compilers.
6. It would not be necessary to use `std::kill_dependency()` calls in most cases. That said, use of `std::kill_dependency()` might at some future point allow the compiler to produce better diagnostics for dubious dependency-chain use cases. For example a compiler might issue a warning for the value-narrowing hazard shown on line 3 of Figure 13, and that diagnostic might be suppressed as shown in Figure 37.
7. More generally, this approach allows annotations to be discarded, as was shown in Figures 20 and 21. However, the `memory_order_consume` loads are still required in order to support DEC Alpha and prevent compiler optimizations that might otherwise destroy the dependency chain.

²⁰ This needs to be re-verified.

8. It would not be necessary to use `[[carries_dependency]]` attributes. However, as with `std::kill_dependency()`, use of something like `[[carries_dependency]]` might produce better diagnostics for dubious use dependency-chain use cases. That said, it might be preferable to substitute either type or object modifiers for attributes, given that use of attributes is not permitted to change the meaning of the program and also that attributes are not yet supported by the C language.
9. With the exception of one use case involving arrays, no changes to the Linux-kernel source code are required. As noted earlier, a patch is available to remove the Linux kernel's dependency on RCU-protected array indexes [22], and this patch was accepted into the v4.2 release of the Linux kernel.
10. This approach has the advantage of implementation experience extending for more than two decades.

In the future, this approach could be augmented by attributes, type modifiers, object modifiers, or other markings to allow more elaborate dependency chains to be created on the one hand, and to improve the compiler's ability to emit diagnostics for dubious uses of dependency chains on the other.

7.10 Draft Wording for Restricted Dependency Chains

Because bit manipulations on pointers results in undefined behavior, the wording for this restricted-dependency-chain approach is considerably simpler than might be expected.²¹ Although there are well-known and heavily used algorithms that depend on setting tag bits on pointers, such algorithms must live outside of the confines of the standard even without dependency chains. This approach has the advantage that dependencies are then carried only through pointer types, which greatly simplifies the specification.

²¹ Kudos to Jens Maurer for pointing this out.

1.10p11 of the most recent draft of the standard[29], is replaced with the following:

An evaluation A *carries a dependency* to an evaluation B of pointer type if

- B is the invocation of an assignment operator and A is the right-hand operand of this operator, or
- B is the invocation of a cast operator that is not a `reinterpret_cast` and A is the operand, or²²
- B is a parameter to a function and A is the value of the corresponding argument, or
- A is the value returned by a function and B is the returned value in the calling context, or
- B is an additive operator, A is an operand to this operator, and the other operand is of integral type.
- B is the prefix `*` dereference operation and A is its operand, or
- B is the `->` class-member access operator and A is its left-hand operand, or
- B is the `?:` operator and A is the operand selected by that operator's condition, or
- B is the `,` operator and A is its rightmost operand, or
- A writes a scalar object or bit-field M, B reads the value written by A from M, and A is sequenced before B, or
- for some evaluations X and Y, X is the unary `&` operator, A is that operator's operand, X carries a dependency to Y, B is the unary `*` operator, and Y is that operator's operand, or
- for some evaluation X, A carries a dependency to X, and X carries a dependency to B.

²² The intent is that casts that leave pass through the bits carry a dependency, while those that affect the pointer value do not.

An evaluation A carries a dependency to a side-effect C if

- C is the side effect resulting from an assignment operator, and A is either operand of this operator, or
- C is the side effect resulting from an atomic read-modify-write operation, A is its first argument, and the corresponding non-atomic operation would carry a dependency, or
- C is the side effect resulting from an `atomic_store()` or `atomic_store_explicit()` function, and A is the function's second argument, or
- C is the side effect resulting from an `atomic_exchange()` or `atomic_exchange_explicit()` function, and A is the function's second argument, or
- C is the side effect resulting from an `atomic_compare_exchange_weak()`, `atomic_exchange_weak_explicit()`, `atomic_compare_exchange_strong()`, or `atomic_exchange_strong_explicit()` function, and A is the function's third (new-value) argument.
- C is the side effect resulting from an atomic read-modify-write member function, A is the corresponding object, and the corresponding non-atomic operation would carry a dependency, or
- C is the side effect resulting from a `store()` or `store_explicit()` member function, and A is the member function's first argument, or
- C is the side effect resulting from an `exchange()` or `exchange_explicit()` member function, and A is the member function's first argument, or
- C is the side effect resulting from a `compare_exchange_weak()`, `exchange_weak_explicit()`,

`compare_exchange_strong()`, or `exchange_strong_explicit()` member function, and A is the member function's second (new-value) argument.

An evaluation A carries a dependency to a memory load D if

- D is the memory load resulting from an `atomic_load()` or `atomic_load_explicit()` function, and A is the function's first argument, or
- D is the memory load resulting from an atomic read-modify-write operation, A is its first argument, and the corresponding non-atomic operation would carry a dependency, or
- D is the memory load resulting from an atomic read-modify-write member function, A is the corresponding object, and the corresponding non-atomic operation would carry a dependency.

If a pointer is subjected to a series of comparison operators that enables exact determination of its value, then no dependency is carried to that pointer. *Note:* To carry out comparisons without destroying the dependency chain, use the `std::pointer_cmp_eq_dep()`, `std::pointer_cmp_ne_dep()`, `std::pointer_cmp_gt_dep()`, `std::pointer_cmp_ge_dep()`, `std::pointer_cmp_lt_dep()`, or `std::pointer_cmp_le_dep()` intrinsics.

Note: `std::kill_dependency()` is not mentioned above, so if A is its argument and B its result, no dependency is carried from A to B.

Note: The intent of these rules is to enable compilers to carry out their normal optimizations, while still permitting developers to rely on the common dependency-ordering use cases.

Note: Although the `[[carries_dependency]]` attribute is no longer needed to specify that a dependency chain exists, this attribute can help the implementation provide higher-quality diagnostics. For example, doing an equality comparison to a non-NULL pointer that is dereferenced could result in a warning diagnostic. Such a warning would make the user aware that no dependency was carried to that pointer.

1.10p12 of the most recent draft is updated by inserting the phrase “of pointer type”, resulting in the following:

An evaluation A is *dependency-ordered before* an evaluation B if

- A performs a release operation on an atomic object M of pointer type, and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A, or
- for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.

7.11 Storage Class

This approach, similar to that suggested by Lawrence Crowl in response to the approach described in Section 7.6, uses a new storage class to indicate where dependencies need to be carried. This section also applies lessons learned in discussions of the other approaches, particularly those of Sections 7.3, 7.5 and 7.9. In particular, this approach retains the restriction called out in Section 7.9, namely allowing dependencies to be carried only through pointer-like types, including restricted operations on `intptr_t` and `uintptr_t`.²³

²³ JF Bastien points out that in C++ this could be implemented as a class that is special in a manner similar to the atomic classes. One example of the special handling required is to preserve dependencies past successful pointer equality

The idea is to provide a storage class `_Carries_dependency` that indicates that the modified local variable, struct/union field, function arguments, function parameters, or return value carries a dependency. This is similar to the existing `[[carries_dependency]]` attribute, but is usable by C (which lacks attributes) as well as by C++, and which, unlike `[[carries_dependency]]`, is permitted to change the program’s semantics. This last is important because it provides greater flexibility in avoiding unsolicited (and costly) memory-fence instructions.

This approach results in the code shown in Figures 24 and 25.

This approach uses strict dependencies rather than semantic dependencies, which means that the implementation will need to emit explicit memory-fence instructions in cases where the dependencies would not be preserved by the underlying hardware. However, high quality-of-implementation compilers would be expected to provide options to issue warnings when such instructions were emitted, allowing the developer to modify the code to avoid these expensive instructions. In addition, high quality-of-implementation compilers are expected to carry dependencies past things like successful equality comparisons with no performance degradation. The operations carrying dependencies are exactly those shown for the *carries a dependency* relation shown in 1.10p11 of the standard [29].

Assignment from an object with `_Carries_dependency` storage class to an object not of this storage class would act as an implicit `std::kill_dependency()`. This is useful when protection of a given object is shifted from a dependency chain to a lock or reference count. However, high quality-of-implementation compilers would be expected to provide options to issue warnings when such an implicit `std::kill_dependency()` was encountered, which would allow developers to enforce coding-style rules requiring explicit `std::kill_dependency()`, if desired.

A high quality-of-implementation compiler would also be expected to provide options to enable warn-

comparisons. However, a C-language implementation would require something resembling a storage class, so this proposal starts at that point.

ings to be issued when a dependency chain was headed by something other than a `memory_order_consume` load or an object with the `_Carries_dependency` storage class. A cast could be used to suppress these warnings, which could be provided as `std::unkill_dependency()` if experience indicates that this is useful. This `std::unkill_dependency()` is likely to be useful to enable common code that traverses data structures using dependencies in some cases and using mutual exclusion in other cases.

The new `_Carries_dependency` storage class may appear with `register`, `static`, `thread_local`, and `extern`. If `_Carries_dependency` appears in one declaration of a variable, it shall appear in all other declarations of that entity. The `_Carries_dependency` storage class can be applied only to the names of variables, functions, formal parameters, and structure/union fields. Only variables of scalar or pointer type may be marked as `_Carries_dependency`.²⁴

Here are use cases for the storage-class combinations:

- **register `_Carries_dependency`:** A local variable that carries a dependency that should also be stored in a machine register.
- **static `_Carries_dependency`:** A static local variable that carries a dependency. Such a variable is presumably subject to some sort of mutual exclusion mechanism so as to avoid inter-thread access.
- **static `thread_local _Carries_dependency`:** A static thread-local variable that carries a dependency.
- **extern `_Carries_dependency`:** A variable shared among translation units that carries a dependency. Such a variable is presumably subject to some sort of mutual exclusion mechanism so as to avoid inter-thread access.
- **extern `thread_local _Carries_dependency`:** A thread-local variable shared among translation units that carries a dependency.

²⁴ But if you have a valid use case for an aggregate construct that carries a dependency, please let us know.

- **thread_local `_Carries_dependency`:** A thread-local variable that carries a dependency.

Again, this `_Carries_dependency` storage class can only be applied to pointer-like types, with only restricted operations permitted on `intptr_t` and `uintptr_t` (see Section 7.9.1 for details).

This approach has the following advantages and disadvantages:

1. The implementation is likely to be simpler in that dependencies need not be traced. Instead, only computations involving objects of the `_Carries_dependency` storage class need be handled specially. This eliminates unnecessary restriction of optimizations on unintentional dependency chains.
2. The implicit `std::kill_dependency()` means that many irrelevant dependency chains are pruned by default, so that fewer explicit `std::kill_dependency()` invocations are required.
3. Attributes need not be added to the C language.
4. No modifications to the type system are required.
5. Applying this to the Linux kernel would require significant marking of variables carrying dependencies, however, the Linux kernel is expected to use the approach called out in Section 7.9. Longer term, some portions of the Linux kernel might incrementally transition to this object-modifier approach, especially if doing so provides significant diagnostic benefits.
6. The strict dependencies should permit tools based on formal methods to successfully analyze dependency chains. High quality tooling might well encourage existing RCU-using projects to adopt this storage-class-based approach.

7.12 Evaluation

This evaluation starts by enumerating the different audiences that any change to `memory_order_consume`

must address (Section 7.12.1) and then compares the various proposals based on the perceived viewpoints of these audiences (Section 7.12.2).

7.12.1 Audiences

The main audiences for any change to `memory_order_consume` include standards committee members, compiler implementers, formal-methods researchers, developers intending to write new code, and developers working with existing RCU code. The Linux kernel community is of course a notable example of this last category.

Standards committee members would like a clean and non-intrusive change to the standard. They would of course also like solutions minimizing the number and vehemence of complaints from the other audiences, or, failing that, reducing the complaints to a tolerable noise level.

Compiler implementers would like a mechanism that fits nicely into current implementations, which does much to explain their satisfaction with the approach of strengthening `memory_order_consume` to `memory_order_acquire`. In particular, they would like to avoid unbounded tracing of dependencies, and would prefer minimal constraints on their ability to apply time-honored optimizations.

Formal-methods researchers would like a definition of `memory_order_consume` that fits into existing theoretical frameworks without undue conceptual violence. Of particular concern is any need to deal with counter-factuals, in other words, any need to reason not only about values of variables required for the solution of a given litmus test, but also about other unrelated values for these variables. As such, counter-factuals are the rock upon which otherwise attractive approaches involving semantic dependency have foundered.²⁵ Some practitioners might wonder why the opinion of formal-methods researchers should be given any weight at all, and the answer to this question is that it is the work of formal-methods researchers that provides us the much-needed tools that we need to analyze both the memory-ordering

specification itself as well as programs using that specification.

Developers writing new code need something that expresses their algorithm with a minimum of syntactic saccharine, that is easy to learn, and that is easy to maintain. For example, one of the weaknesses of the current standards' definition of `memory_order_consume` is the need to sprinkle large numbers of `std::kill_dependency()` calls throughout one's code. In short, developers would like it to be easy to write, analyze, and maintain code that uses dependency ordering.

Developers with existing RCU code have the same desires as do developers writing new code, but are also very interested in minimizing the code churn required to adhere to the standard.

The challenge if of course to find a proposal that addresses the viewpoints of all of these audiences. As we will see in the next session, this is not easy. However, there is some hope that the approach presented in Section 7.9 might suffice.

7.12.2 Comparison

A summary comparison of the proposals is shown in Table 1.

The dependency type can either be “dep” for normal dependency, “rdep” for the restricted dependencies discussed in Section 7.9, “sdep” for (global) semantic dependency, or “lsdep” for local semantic dependency.²⁶ Variable, formal-parameter, and return-value marking can either be type-based (“T”), storage-class-based (“S”), attribute-based (“A”), or not required (“ ”). Beginning-of-chain handling can either require explicit indication to which quantities dependencies must be carried (“D”) or nothing (“ ”). End-of-chain handling can either require an explicit `std::kill_dependency` (“K”), an implicit `std::kill_dependency` (“k”), explicit designation of

²⁵ That said, Alan Jeffries is making another attempt to come up with a suitable formal definition of semantic dependency.

²⁶ Recall that a local semantic dependency remains a dependency even if the `memory_order_consume` load at its head can return only a single value. In contrast, a global semantic dependency remains a dependency only if more than one value can appear at the end of the chain. Therefore, optimizations based on global full-program analysis can break a global semantic dependency but can break neither a local semantic dependency nor a normal dependency.

	Dependency Type	Object Marking	Formal-Parameter Marking	Return-Value Marking	Beginning-Of-Chain Handling	End-Of-Chain Handling	Dependency Tracing Required	C Attribute Support Required	Diagnostic Capability	Unsolicted Memory Fences	Linux-Kernel Compatibility	Suitable for Formal Verification
C11 / C++11	dep		A	A		K	Y	Y	n	Y	N	
Type-Based Designation of Dependency Chains With Restrictions (Section 7.2)	lsdep	T	T	T		k			?	Y	N	N
Type-Based Designation of Dependency Chains (Section 7.3)	dep	T	T	T		k			?	Y	N	
Whole-Program Option (Section 7.4)	sdep								N			N
Local-Variable Restriction (Section 7.5)	dep		A	A		k		Y	?	Y	N	
Mark Dependency-Carrying Local Variables (Section 7.6)	dep	A	A	A		k		Y	?	Y	N	
Explicitly Tail-Marked Dependency Chains (Section 7.7)	dep		A	A		Dk	y	Y	N	Y	N	?
Explicitly Head-Marked Dependency Chains (Section 7.8)	dep		?	?	D	k	y	Y	N	Y	N	?
Restricted Dependency Chains (Section 7.9)	rdep								N			N
Storage Class (Section 7.11)	rdep	S	S	S						Y	N	

Marking: “A”: attribute, “S”: Object, “T”: type.

Beginning of chain: “D”: explicit designation.

End of chain: “D”: explicit designation, “k/K”: implicit/explicit `std::kill_dependency`

Dependency tracking required: “y”: only for marked chains, “Y”: always.

Diagnostic capability: “n”: incomplete, “N”: very limited, “?”: could be provided.

Table 1: Comparison of Consume Proposals

dependency (“D”), or nothing (“ ”).²⁷ Dependency tracking might be required for all chains (“Y”), explicitly designated chains (“y”), or not required at all (“ ”). C-language `[[carries_dependency]]` support might be required (“Y”) or not (“ ”). Diagnostic capability is difficult to evaluate for many of the proposals, but for some proposals it is limited (“n”), for others quite difficult (“N”), and for still others too early to tell (“?”). Some proposals inflict unsolicited memory fences on developers (“Y”), and others do not (“ ”), and some proposals are source-code compatible with the Linux kernel (“ ”), while others are not (“N”). Anything other than pure syntactic dependencies (“dep”) are quite challenging for current formal-verification tools, resulting in most proposals getting (“N”) or (“?”).

The ideal proposal would have dependency type “dep” (thus making it easier to model dependency ordering and making it unnecessary for developer to have to outwit full-program optimizations), no need for variable, formal-parameter, or return-value marking (thus minimizing changes required for existing RCU code), implicit “do the right thing” end-of-chain handling,²⁸ (thus minimizing the need for whack-a-mole source-code markups), no need for dependency tracking (thus making it easier to implement), no need for C-language support for the `[[carries_dependency]]` attribute (thus minimizing changes to the C standard), good diagnostic capabilities, refrain from supplying unsolicited memory fences, be source-code compatible with the Linux kernel, and lend itself well to analysis by formal-verification tools.

However, there appears to be no such proposal possible, so the recommended approach is to use the restricted dependency chains discussed in Section 7.9 for large existing code bases and the storage-class approach discussed in Section 7.11 for new code and perhaps also as a future direction for large existing code bases.

²⁷ Variables that go out of scope always have any dependency chain implicitly killed.

²⁸ Perhaps implemented by a careful choice of exactly which operators carry dependencies in which situations.

8 Summary

This document has analyzed Linux-kernel use of dependency ordering and has laid out the status-quo interaction between the Linux kernel and pre-C11 compilers. It has also put forward some possible ways of building towards a full implementation of C11’s and C++11’s handling of dependency ordering. It calls out some weaknesses in C11’s and C++11’s handling of dependency ordering and offers some alternatives. Finally, it recommends one approach (Section 7.9) for large existing code bases and a second approach (Section 7.11) for new code bases. Over time, it is hoped that the availability of diagnostic tools will motivate migration to the second of these two approaches.

References

- [1] ALGLAVE, J., MARANGET, L., PAWAN, P., SARKAR, S., SEWELL, P., WILLIAMS, D., AND NARDELLI, F. Z. PPCMEM/ARMMEM: A tool for exploring the POWER and ARM memory models. <http://www.cl.cam.ac.uk/~pes20/ppc-supplemental/pldi105-sarkar.pdf>, June 2011.
- [2] ALGLAVE, J., MARANGET, L., AND TAUTSCHNIG, M. Herding cats: Modelling, simulation, testing, and data-mining for weak memory. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation* (New York, NY, USA, 2014), PLDI ’14, ACM, pp. 40–40.
- [3] ARM LIMITED. *ARM Architecture Reference Manual: ARMv7-A and ARMv7-R Edition*, 2010.
- [4] BOEHM, H. J. Space efficient conservative garbage collection. *SIGPLAN Not.* 39, 4 (Apr. 2004), 490–501.
- [5] BONZINI, P., AND DAY, M. RCU implementation for Qemu. <http://lists.gnu.org/archive/html/qemu-devel/2013-08/msg02055.html>, August 2013.

- [6] DALTON, M. *THE DESIGN AND IMPLEMENTATION OF DYNAMIC INFORMATION FLOW TRACKING SYSTEMS FOR SOFTWARE SECURITY*. PhD thesis, Stanford University, 2009. Available: http://cs1.stanford.edu/~christos/publications/2009.michael_dalton.phd_thesis.pdf [Viewed March 9, 2010].
- [7] DESNOYERS, M. [RFC git tree] userspace RCU (urcu) for Linux. <http://liburcu.org>, February 2009.
- [8] DESNOYERS, M., MCKENNEY, P. E., STERN, A., DAGENAIS, M. R., AND WALPOLE, J. User-level implementations of read-copy update. *IEEE Transactions on Parallel and Distributed Systems* 23 (2012), 375–382.
- [9] GRISENTHWAITE, R. *ARM Barrier Litmus Tests and Cookbook*. ARM Limited, 2009.
- [10] HOWARD, P. W., AND WALPOLE, J. A relativistic enhancement to software transactional memory. In *Proceedings of the 3rd USENIX conference on Hot topics in parallelism* (Berkeley, CA, USA, 2011), HotPar’11, USENIX Association, pp. 1–6.
- [11] HOWARD, P. W., AND WALPOLE, J. Relativistic red-black trees. *Concurrency and Computation: Practice and Experience* (2013), n/a–n/a.
- [12] INTEL CORPORATION. *A Formal Specification of Intel Itanium Processor Family Memory Ordering*, 2002. Available: <http://developer.intel.com/design/itanium/downloads/251429.htm> <ftp://download.intel.com/design/Itanium/Downloads/25142901.pdf> [Viewed: January 10, 2007].
- [13] INTERNATIONAL BUSINESS MACHINES CORPORATION. *Power ISA Version 2.07*, 2013.
- [14] KANNAN, H. Ordering decoupled metadata accesses in multiprocessors. In *MICRO 42: Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture* (New York, NY, USA, 2009), ACM, pp. 381–390.
- [15] MCCARTHY, J. Recursive functions of symbolic expressions and their computation by machine, part i. *Commun. ACM* 3, 4 (Apr. 1960), 184–195.
- [16] MCKENNEY, P. E. Read-copy update (RCU) usage in Linux kernel. Available: <http://www.rdrop.com/users/paulmck/RCU/linuxusage/rculocktab.html> [Viewed January 14, 2007], October 2006.
- [17] MCKENNEY, P. E. What is RCU? part 2: Usage. Available: <http://lwn.net/Articles/263130/> [Viewed January 4, 2008], January 2008.
- [18] MCKENNEY, P. E. The RCU API, 2010 edition. <http://lwn.net/Articles/418853/>, December 2010.
- [19] MCKENNEY, P. E. Validating memory barriers and atomic instructions. <http://lwn.net/Articles/470681/>, December 2011.
- [20] MCKENNEY, P. E. *Is Parallel Programming Hard, And, If So, What Can You Do About It?* kernel.org, Corvallis, OR, USA, 2012.
- [21] MCKENNEY, P. E. Structured deferral: synchronization via procrastination. *Commun. ACM* 56, 7 (July 2013), 40–49.
- [22] MCKENNEY, P. E. [PATCH tip/core/rcu 1/4] mce: Stop using array-index-based RCU primitives. [PATCHtip/core/rcu1/4]mce: Stopusingarray-index-basedRCUprimitives, May 2015.
- [23] MCKENNEY, P. E., PURCELL, C., ALGAE, SCHUMIN, B., CORNELIUS, G., QWERTYUS, CONWAY, N., SBW, BLAINSTER, RUFUS, C., ZOICON5, ANOME, AND EISEN, H. Read-copy update. <http://en.wikipedia.org/wiki/Read-copy-update>, July 2006.
- [24] MCKENNEY, P. E., AND SLINGWINE, J. D. Read-copy update: Using execution history to solve concurrency problems. In *Parallel and Distributed Computing and Systems* (Las Vegas, NV, October 1998), pp. 509–518.

- [25] MCKENNEY, P. E., AND STERN, A. Axiomatic validation of memory barriers and atomic instructions. <http://lwn.net/Articles/608550/>, August 2014.
 - [26] MCKENNEY, P. E., AND WALPOLE, J. What is RCU, fundamentally? Available: <http://lwn.net/Articles/262464/> [Viewed December 27, 2007], December 2007.
 - [27] MICHAEL, M. M. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems* 15, 6 (June 2004), 491–504.
 - [28] ROSSBACH, C. J., HOFMANN, O. S., PORTER, D. E., RAMADAN, H. E., BHANDARI, A., AND WITCHEL, E. TxLinux: Using and managing hardware transactional memory in an operating system. In *SOSP'07: Twenty-First ACM Symposium on Operating Systems Principles* (October 2007), ACM SIGOPS. Available: <http://www.sosp2007.org/papers/sosp056-rossbach.pdf> [Viewed October 21, 2007].
 - [29] SMITH, R. Working draft, standard for programming language C++. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4527.pdf>, May 2015.
 - [30] VIGUERAS, G., ORDUÑA, J. M., AND LOZANO, M. A Read-Copy Update based parallel server for distributed crowd simulations. *The Journal of Supercomputing* (Apr. 2012).
1. Move Section 7.1, which summarizes the Linux-kernel uses of dependency chains, up to the beginning of Section 7.
 2. Fix some typos in Section 7.4, and add words accounting for DEC Alpha, which must continue to promote `memory_order_consume` loads to `memory_order_acquire` with this section’s approach. Also clarify Jeff Preshing’s conjecture about the lack of optimizations that break semantic dependency chains.

(June 5, 2014.)

- Add Section 7.5, which covers Hans Boehm’s approach, which restricts dependency chains to local variable. Also add Section 7.6, which covers Clark Nelso’s approach, which uses the `[[carries_dependency]]` attribute to mark those local variables that carry dependencies. (July 17, 2014.)
- Mark the paper as a revision of N4036. (July 17, 2014.)
- Add Jeff Preshing, Hans Boehm, and Clark Nelson as authors, and clarify Hans’s proposal in Section 7.5. (August 13, 2014.)
- Add Olivier Giroux’s proposal for explicitly marking the tails of dependency chains as Section 7.7, and add Olivier as author. Update the advantages and disadvantages of Jeff Preshing’s whole-program option in Section 7.4 Add advantages and disadvantages for Hans Boehm’s proposal in Section 7.5 and for Clark Nelson’s proposal in Section 7.6. (August 23, 2014.)
- Update Olivier Giroux’s explicit dependency-chain marking proposal in Section 7.7 based on feedback from Olivier. (August 24, 2014.)
- Add a section header (Section 4.1.1) for the dependency-chain rules for 2014 GCC implementations. Add Section 4.1.3 laying out the simpler rules for managing dependency chains in the older 1990s Sequent C implementations. Add a footnote to Section 7.6 recording Lawrence

Change Log

This paper first appeared as **N4026** in May of 2014. Revisions to this document are as follows:

- Add a suggestion to Section 7.1 that developers provide diagnostics for singleton arrays when using RCU-protected indexes. (May 28, 2014.)
- Apply changes from Torvald Riegel feedback:

Crowl's preference for object modifiers instead of either attributes or type modifiers. Add words to Section 7.7 noting that `atomic_signal_fence()` or the Linux kernel's `barrier()` can be used to suppress code-motion optimizations that might otherwise prevent fixing up tail-marked dependency chains that were broken by code-motion optimizations. (October 1, 2014.)

- Add Section 3.4, which describes operators that act as the last link in the Linux kernel's dependency chains, including the ambiguous role of the `->` operator. (October 1, 2014.)
- Update plots of RCU API usage. (October 5, 2014.)
- Flesh out Section 4.1.2, which lays out a short history of dependency-chain management rules for the Linux kernel. Numerous minor clarifications and corrections throughout the document. (October 5, 2014.)
- Add “and” before final author's name and email address. Also add a pointer back to N4036. (October 5, 2014.)
- Add a brain-dead build script. (October 5, 2014.)
- Add Section 7.12, which has the beginnings of an evaluation of the various proposals. (October 5, 2014.)

The paper was then published as **N4215**, which was revised as follows:

- Add verbiage to Sections 7.2 and 7.12.2 differentiating between various forms of syntactic and semantic dependencies. (November 11, 2014.)
- Add a now-deleted section suggesting a C keyword instead of the C++ `[[carries_dependency]]` attribute. This section also suggests that combining ideas from several proposals might be helpful. (November 11, 2014.)
- Add Section 7.8 which contains a first attempt to describe Olivier Giroux's head-marked dependency chains. (November 11, 2014.)

- Add example code in Figures 15 and 16. Add similar figures to the proposals showing how they mark the dependency chains in the sample code. (November 11, 2014.)

At this point, the paper was published as **N4321**, which was revised as follows:

- Add a warning against using operands to `&` and `|` that leave only one bit set (or, respectively, cleared) to Section 4.1.1. (April 9, 2015.)
- Update plots of RCU API usage. (April 19, 2014.)
- Add Section 7.9, which restricts dependency chains with the goal of allowing unmarked dependency chains while avoiding restricting compiler optimizations. One important restriction is the elimination of RCU-protected array indexes. (April 19, 2014.)
- Add verbiage to Section 3.2 noting that the `&` operator is sometimes used to find the beginning of a power-of-two aligned structure. (April 19, 2014.)
- Apply feedback from Linus Torvalds to Section 7.9. (May 20, 2015.)
- Apply feedback from Linus Torvalds to Section 7.9. (May 20, 2015.)
- Apply feedback from Torvald Riegel to Section 7.9. (May 20, 2015.)
- Don't allow integers to carry dependencies into or out of unmarked functions in Section 7.9. (May 21, 2015.)
- Update Section 7.9 to note that assignments cannot extend a dependency chain from one thread to another, and that assignments cannot extend an `intptr_t`- or `uintptr_t`-based dependency chain through a shared variable, even within a given thread. (May 21, 2015.)
- Update Section 7.9 to note that atomic operations can extend dependency chains in the same way that their non-atomic counterparts can. (May 21, 2015.)

- Update Section 7.9 to explicitly call out the fact that stores and subsequent loads can extend dependency chains. (May 21, 2015.)
 - Update Section 7.9 to explicitly state that undefined behavior terminates dependency chains, and that if only one value of a pointer avoids undefined behavior, any dependency chains through that pointer at that point are terminated. (May 22, 2015.)
 - Update Section 7.9 to document the difficulties stemming from value-specialization optimizations. (May 23, 2015.)
 - Update Section 7.9 to note the back-propagation of the dependency-chain-breaking effects of the compiler deducing the exact value of a pointer. (May 27, 2015.)
 - Update Section 7.9 to give more information on how much bit-setting and bit-clearing is too much. (June 8, 2015.)
 - Update Section 7.9 to note that the Linux kernel does not rely on dependency ordering when a statically allocated structure is linked into an RCU-protected structure, but also note that optimizations breaking dependency ordering in this case seem quite dubious. (July 11, 2015.)
 - Update Section 7.9 illustrating dependency-chain rules with examples. (July 11, 2015.)
 - Update Section 7.9 noting that simple loads suffice for `memory_order_consume` loads. (July 11, 2015.)
 - Update Sections 7.7 and 7.8 calling out abstraction challenges for head- and tail-marking approaches. (July 13, 2015.)
 - Add subsections to Section 7.9. (July 13, 2015.)
 - Update Section 7.9 to amplify objections to specialization optimizations applied to heap-based pointers. (July 13, 2015.)
 - Update Section 7.9.1 to note that if a given operation extends a dependency chain, then so does its atomic counterpart. (August 21, 2015.)
 - Add text throughout stating that the patches prohibiting carrying of dependencies through integer variables have been accepted into v4.2 of the Linux kernel (with a very few exceptions). (September 5, 2015.)
 - Add clarifications to Section 7.9, particularly surrounding division of responsibility between the developer and the compiler. (September 5, 2015.)
 - Add diagnostic capability and unsolicited memory fences as criteria in Table 1. (September 14, 2015.)
 - Add Linux-kernel compatibility and formal verification as criteria in Table 1. (September 14, 2015.)
 - Add a storage-class proposal in Section 7.11 and update Table 1 accordingly. (September 17, 2015.)
 - Update Linux-kernel synchronization statistics to v4.2. (September 19, 2015.)
 - Update Section 7.11 to clarify that the storage-class approach retains type restriction: Only pointer-like types, with restrictions on operations on `intptr_t` and `uintptr_t`. (September 21, 2015.)
- At this point, the paper was published as **P0098R0**, which was revised as follows:
- Convert lingering C syntax to C++, as suggested by Thomas Koepp. (October 1, 2015.)
 - Update standards references from n3691 to n4527, as suggested by Jens Maurer. (January 4, 2016.)
 - Add introductory sentences indicating that this is the last version of this, and that the follow-on will be a concrete proposal for handling of `memory_order_consume`.

At this point, the paper was published as
P0098R1.